

とてもたのしい家庭菜園

解説：岸田陸玖

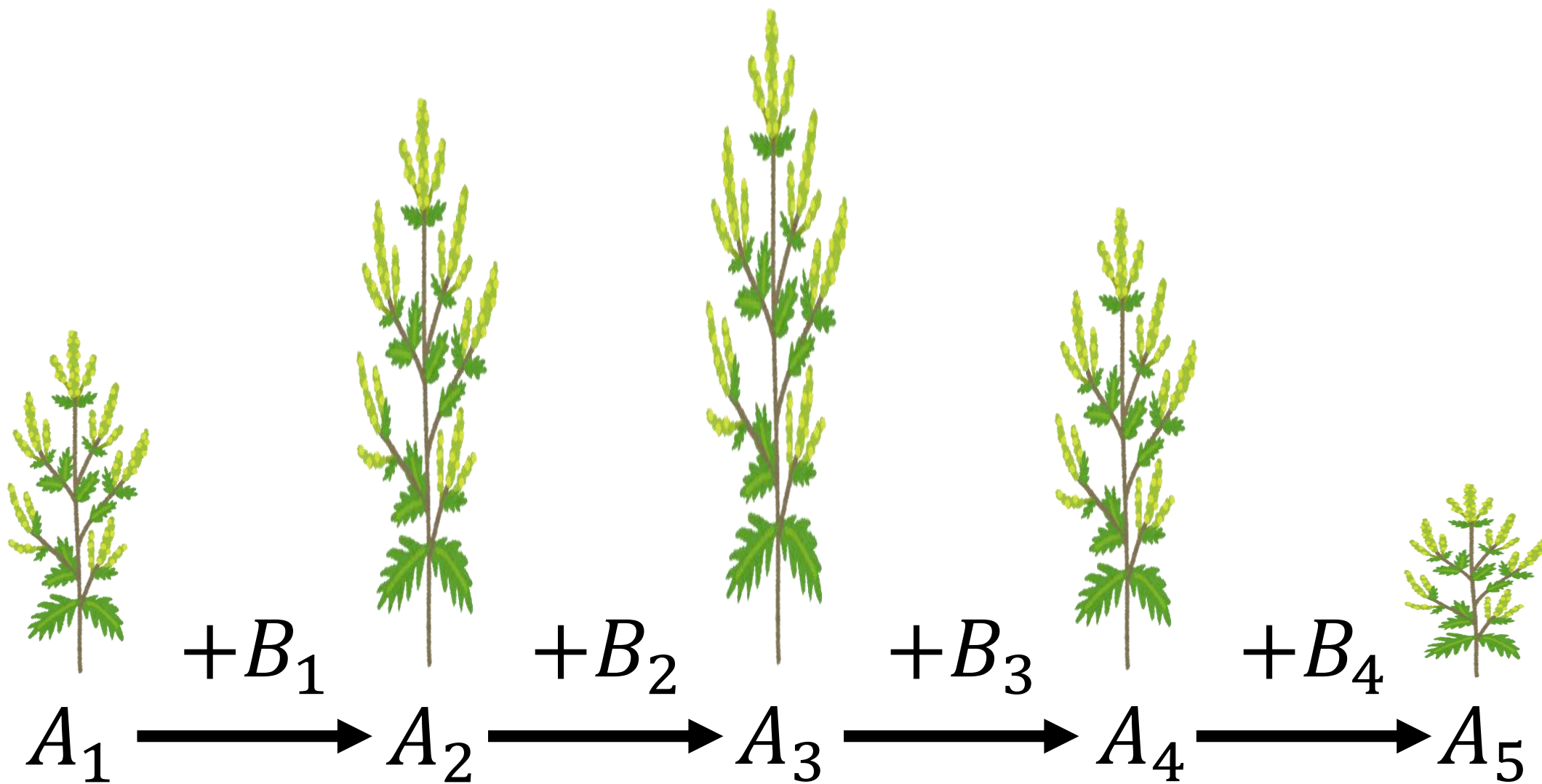
問題概要

- 長さ N の数列 A が与えられる

操作： $A_l, A_{l+1}, \dots, A_r$ に **+1** 加算

条件： $A_1 < \dots < A_k > \dots > A_N$ となる k ($1 \leq k \leq N$) が存在

- 操作の最小回数は？



解法

- $B_i := A_{i+1} - A_i$ ($1 \leq i \leq N - 1$) とする

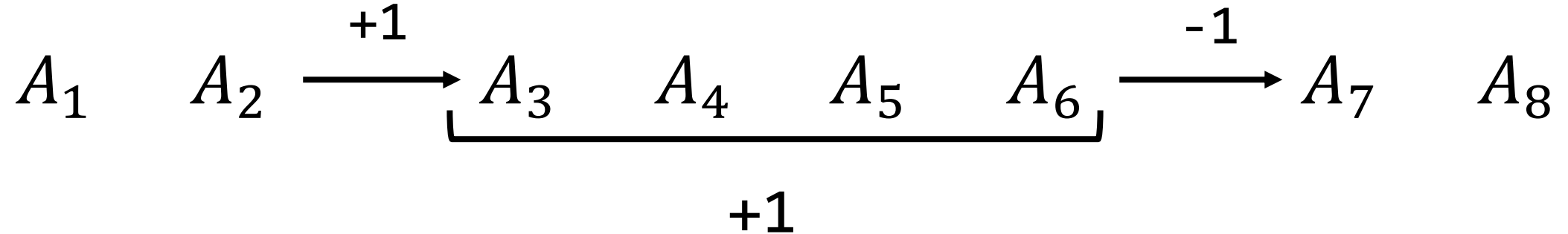
$A_1 < \cdots < A_k > \cdots > A_N$ となる k ($1 \leq k \leq N$) が存在



$B_j \geq 1$ ($1 \leq j \leq k - 1$) かつ $B_j \leq -1$ ($k \leq j \leq N - 1$)
となる k ($1 \leq k \leq N$) が存在

解法

- $B_i := A_{i+1} - A_i$ ($1 \leq i \leq N - 1$) とする



- B_i の値は境界部分のみ変化する

問題概要

- 長さ $N - 1$ の数列 B が与えられる

操作(l, r) : B_l に $+1$ 加算、 B_r に -1 加算 ($1 \leq l < r \leq N - 1$)

操作(l) : B_l に $+1$ 加算 ($1 \leq l \leq N - 1$)

操作(r) : B_r に -1 加算 ($1 \leq r \leq N - 1$)

条件 : $B_j \geq 1$ ($1 \leq j \leq k - 1$) かつ $B_j \leq -1$ ($k \leq j \leq N - 1$)

を満たす k ($1 \leq k \leq N$) が存在

- 操作の最小回数は？

条件： $B_j \geq 1$ ($1 \leq j \leq k-1$) かつ $B_j \leq -1$ ($k \leq j \leq N-1$)
を満たす k ($1 \leq k \leq N$) が存在

- k の値が固定されたときの操作の最小回数を求めたい
- B_j ($1 \leq j \leq k-1$) に着目してみる
- B_j に **-1** 加算すること、
 $B_j \geq 1$ となっている値に **+1** 加算することは不必要
- 操作を**1**回するとある場所を **+1** 加算できるので、
少なくとも $\sum_{j=1}^{k-1} \max(1 - B_j, 0)$ 回の操作が必要

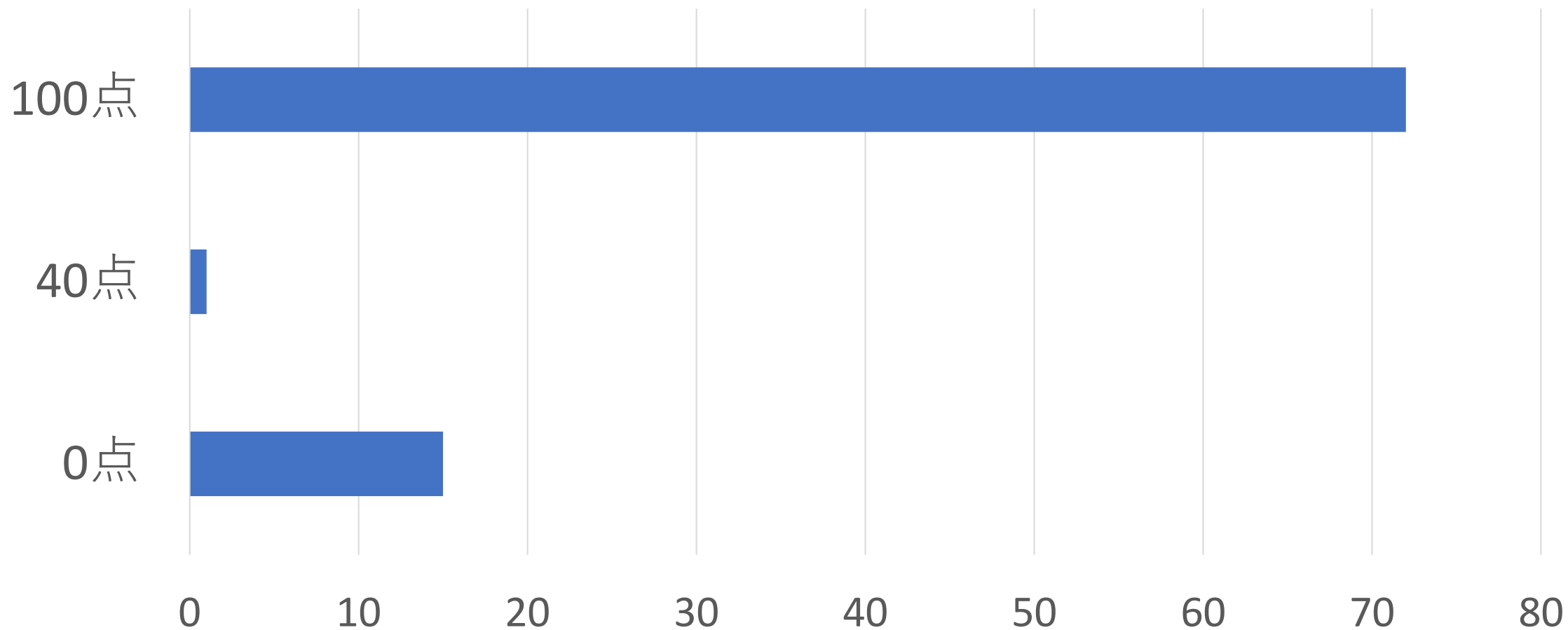
条件： $B_j \geq 1$ ($1 \leq j \leq k-1$) かつ $B_j \leq -1$ ($k \leq j \leq N-1$)
を満たす k ($1 \leq k \leq N$) が存在

- k の値が固定されたときの操作の最小回数を求めたい
- B_j ($k \leq j \leq N-1$) も同様の考察で
少なくとも $\sum_{j=k}^{N-1} \max(1 + B_j, 0)$ 回の操作が必要
- 操作回数の下限は
 $C_k := \max\left(\sum_{j=1}^{k-1} \max(1 - B_j, 0), \sum_{j=k}^{N-1} \max(1 + B_j, 0)\right)$ となる
- 操作(1,r)を使うことで C_k 回の操作で条件を満たすことが可能

解法

- 答えは $\min(C_k ; 1 \leq k \leq N)$
ただし $C_k := \max(\sum_{j=1}^{k-1} \max(1 - B_j, 0), \sum_{j=k}^{N-1} \max(1 + B_j, 0))$
- $L_k := \sum_{j=1}^{k-1} \max(1 - B_j, 0), R_k := \sum_{j=k}^{N-1} \max(1 + B_j, 0)$ とおくと
- $L_k = L_{k-1} + \max(1 - B_{k-1}, 0), R_k = R_{k+1} + \max(1 + B_k, 0)$
- $\Theta(N)$ で計算可能

得点分布



雪玉

[SNOWBALL]

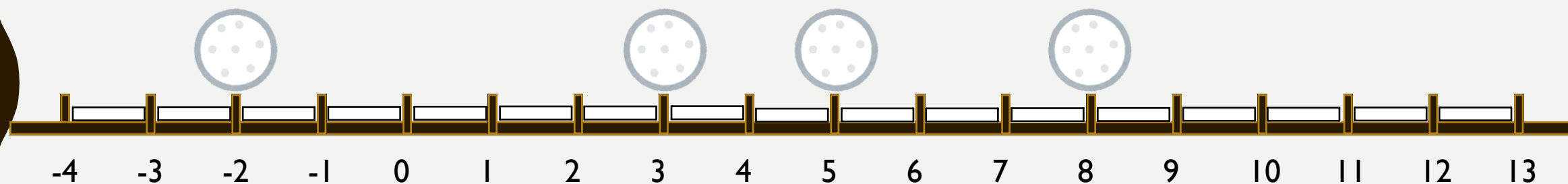
解説：戸高 空

問題概要

- 数直線上に N 個の雪玉がある
- 全体に雪が積もっている

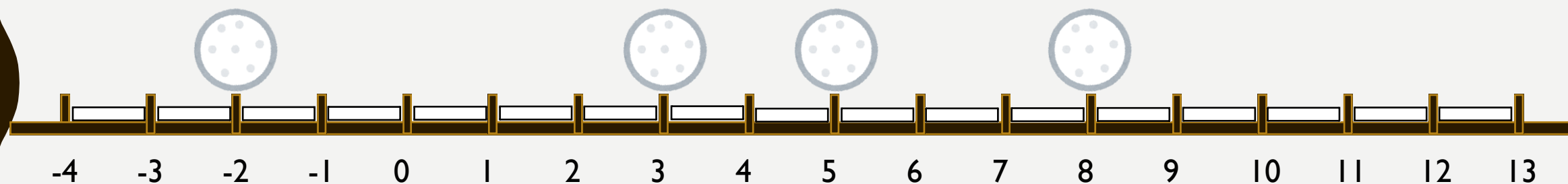
問題概要

- 数直線上にN個の雪玉がある
- 全体に雪が積もっている



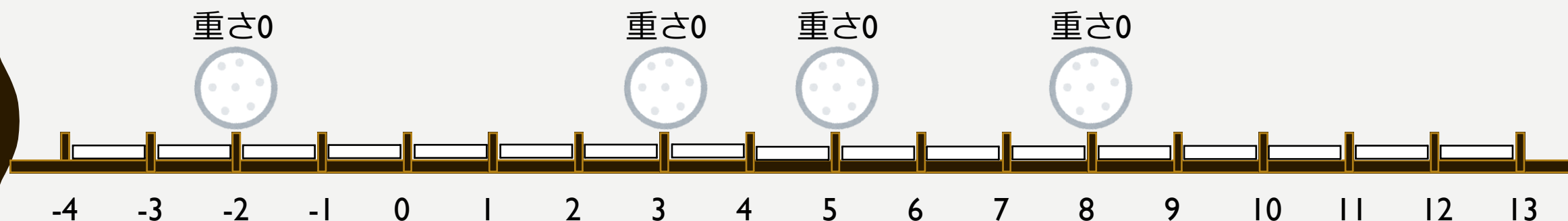
問題概要

- 風が吹いて雪玉が転がるということが Q 回起きる
- j 回目の風ですべての雪玉が w_j だけ転がる
- 雪のある範囲を初めて転がった雪玉の重さが1増える



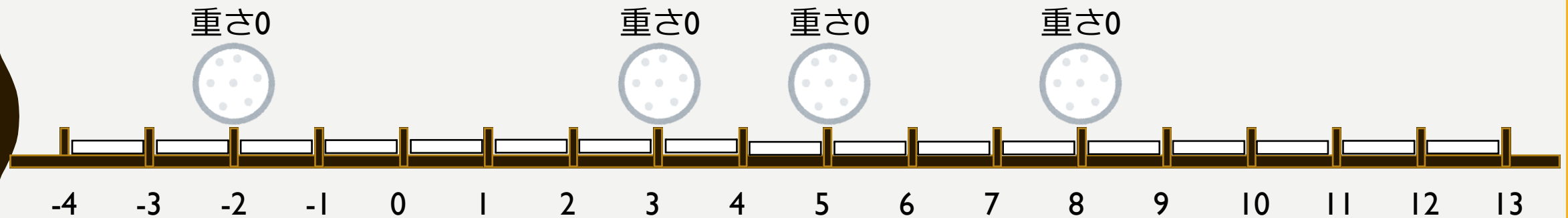
問題概要

- 風が吹いて雪玉が転がるということが Q 回起きる
- すべての雪玉が w_j だけ転がる
- 雪のある範囲を初めて転がった雪玉の重さが1増える



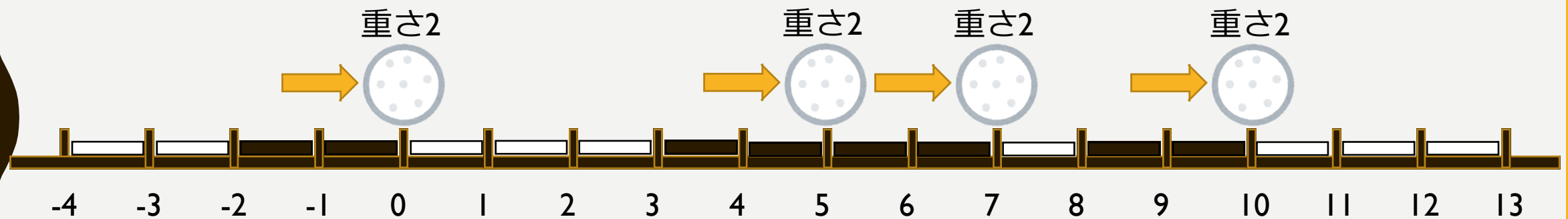
問題概要

- 入力例1



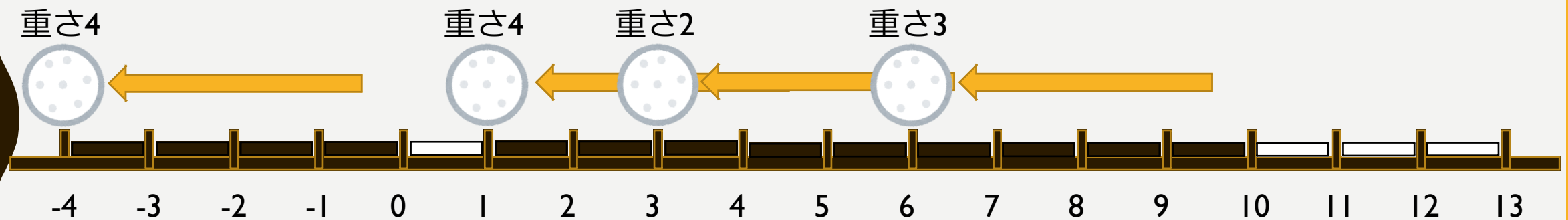
問題概要

- 入力例1
- $W_1 = 2$



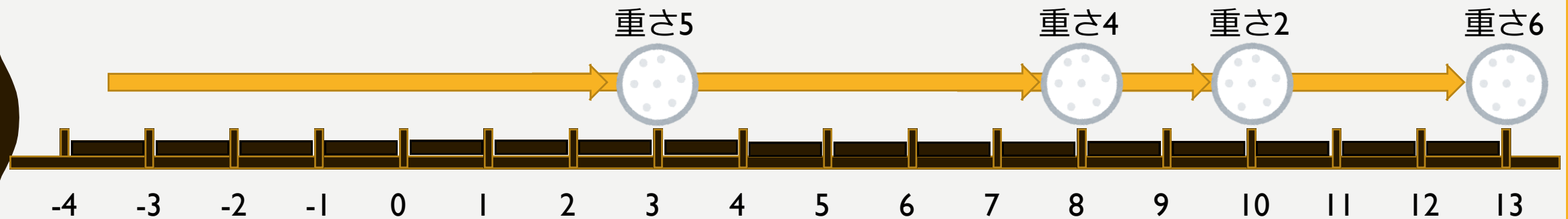
問題概要

- 入力例1
- $W_2 = -4$



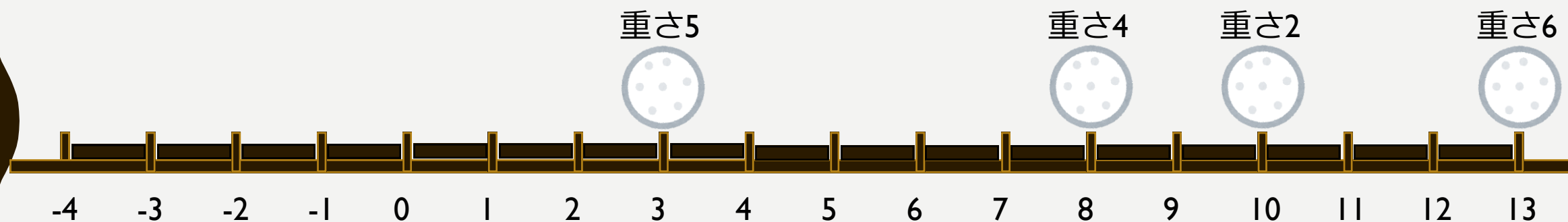
問題概要

- 入力例1
- $W_3 = 7$



問題概要

- Q 回風が吹いた後の各雪玉の重さを求めよ
- $N, Q \leq 200\,000$

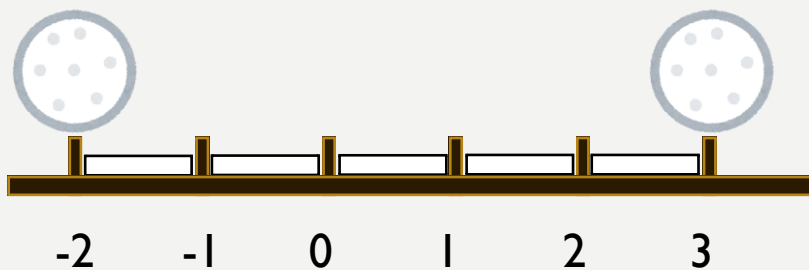


小課題 1 [33点]

- $N, Q \leq 2\,000$

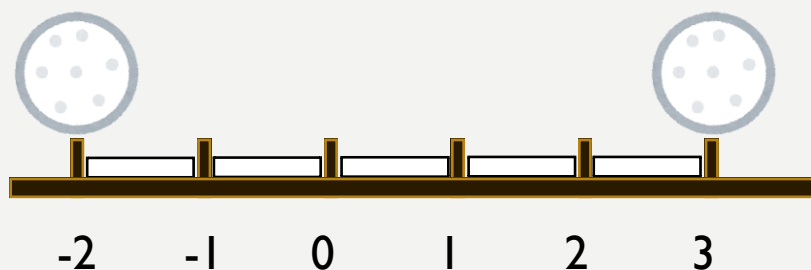
小課題 1 [33点]

- $N, Q \leq 2\,000$
- 最初隣合う2つの雪玉で挟まれた区間を考える



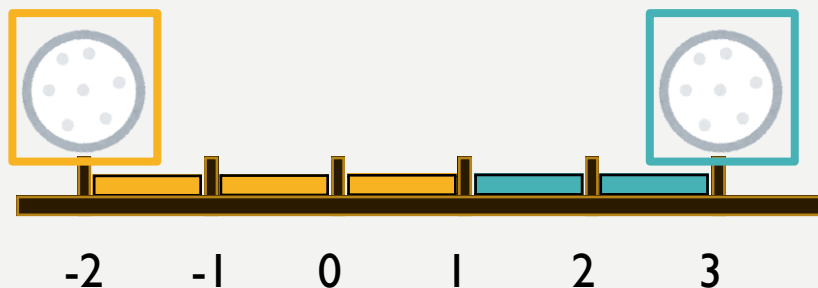
小課題 1 [33点]

- $N, Q \leq 2\,000$
- 最初隣合う2つの雪玉で挟まれた区間を考える
- その区間の雪をもらうのは2つの雪玉のどちらか



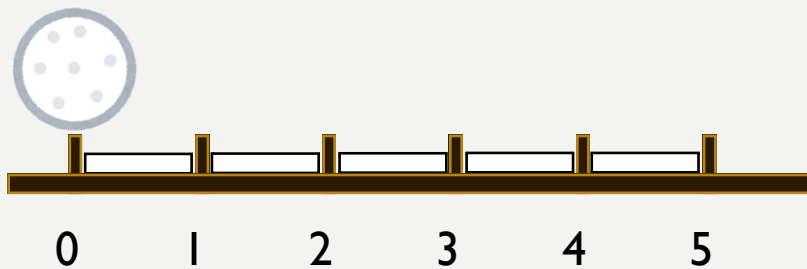
小課題 1 [33点]

- $N, Q \leq 2\,000$
- 最初隣合う2つの雪玉で挟まれた区間を考える
- その区間の雪をもらうのは2つの雪玉のどちらか
- 各区間ごとに独立に左右の雪玉への寄与を計算する



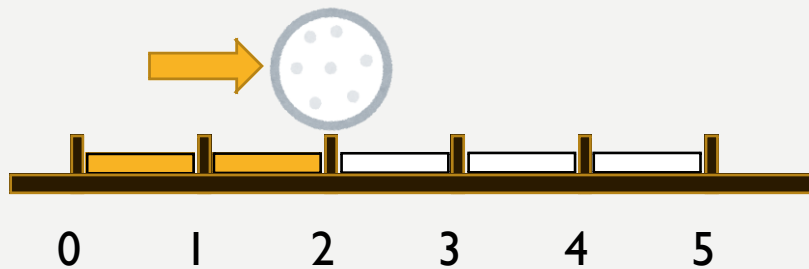
小課題 1 [33点]

- 雪玉の雪の取り方を観察



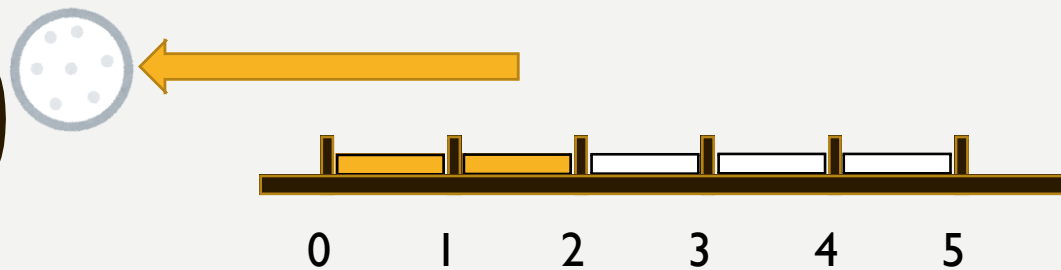
小課題 1 [33点]

- 雪玉の雪の取り方を観察



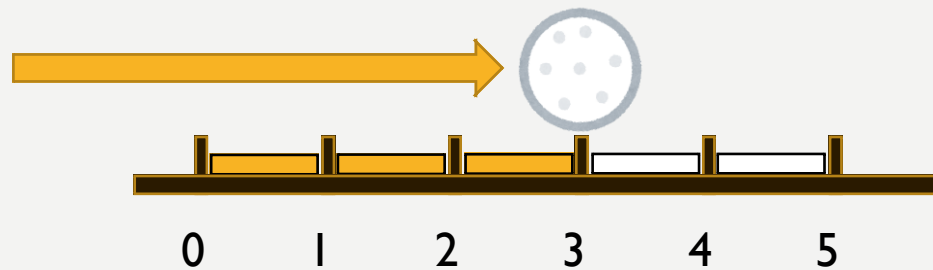
小課題 1 [33点]

- 雪玉の雪の取り方を観察



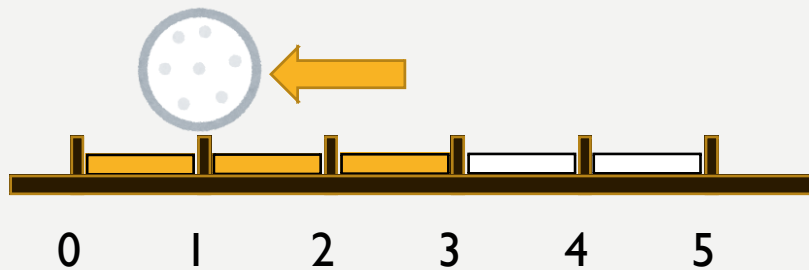
小課題 1 [33点]

- 雪玉の雪の取り方を観察



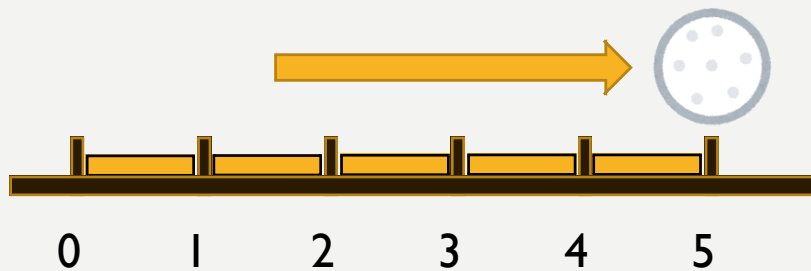
小課題 1 [33点]

- 雪玉の雪の取り方を観察



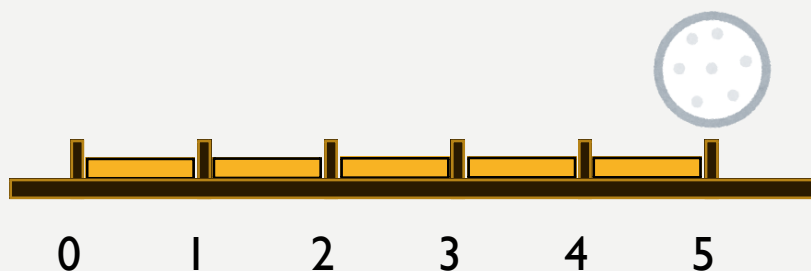
小課題 1 [33点]

- 雪玉の雪の取り方を観察



小課題 1 [33点]

- 最初の座標から右へどれだけ移動したことがあるかが大事
- つまり右への変化量の最大値
- 同じように左への変化量の最大値も大事



小課題 1 [33点]

- 最初の座標からの

(現在の変化量) V

(右への変化量の最大値) R

(左への変化量の最大値) L

の情報を持って風を順番に処理していく

$$V += W_j$$

$$R = \max(R, V)$$

$$L = \max(L, -V)$$

小課題 1 [33点]

- ある区間の左右の雪玉への配分はどうなるか
- 区間の長さを T とする
- $R + L \leq T$ の間は2つの雪玉の転がった範囲は重ならない
- 左の雪玉が R ,右の雪玉が L だけ雪をもらっている
- 区間上に残っている雪は $T - L - R$ である

小課題 1 [33点]

- $R + L > T$ に初めてなる風が重要である
- このとき R が増えたなら左の雪玉
 L が増えたなら右の雪玉が、区間に残っている雪をすべてとる
- そしてこの区間の左右の雪玉への配分はこれで確定する

小課題 1 [33点]

- 具体的には $L + R \leq T$ の状態から R が R' になり $L + R' > T$ となったとする
- 超える前に残っていた雪は $T - L - R$ であるので
- 左の雪玉が $R + T - L - R = T - L$,右の雪玉が L と確定する
- L が増えて超えた場合も同じようにできる

小課題 1 [33点]

- 各区間ごとに $O(Q)$ かけて左右の雪玉への配分を計算する
- $O(NQ)$ で答えが求まる

小課題 2 [67点]

- $N, Q \leq 200\,000$
- 小課題1の解法では同じ計算を何回もしている
- $R + L \leq T$ の間、つまり区間の配分が確定するタイミングまでの処理はすべての区間で共通である
- また区間の長さが小さい順に配分が確定していく

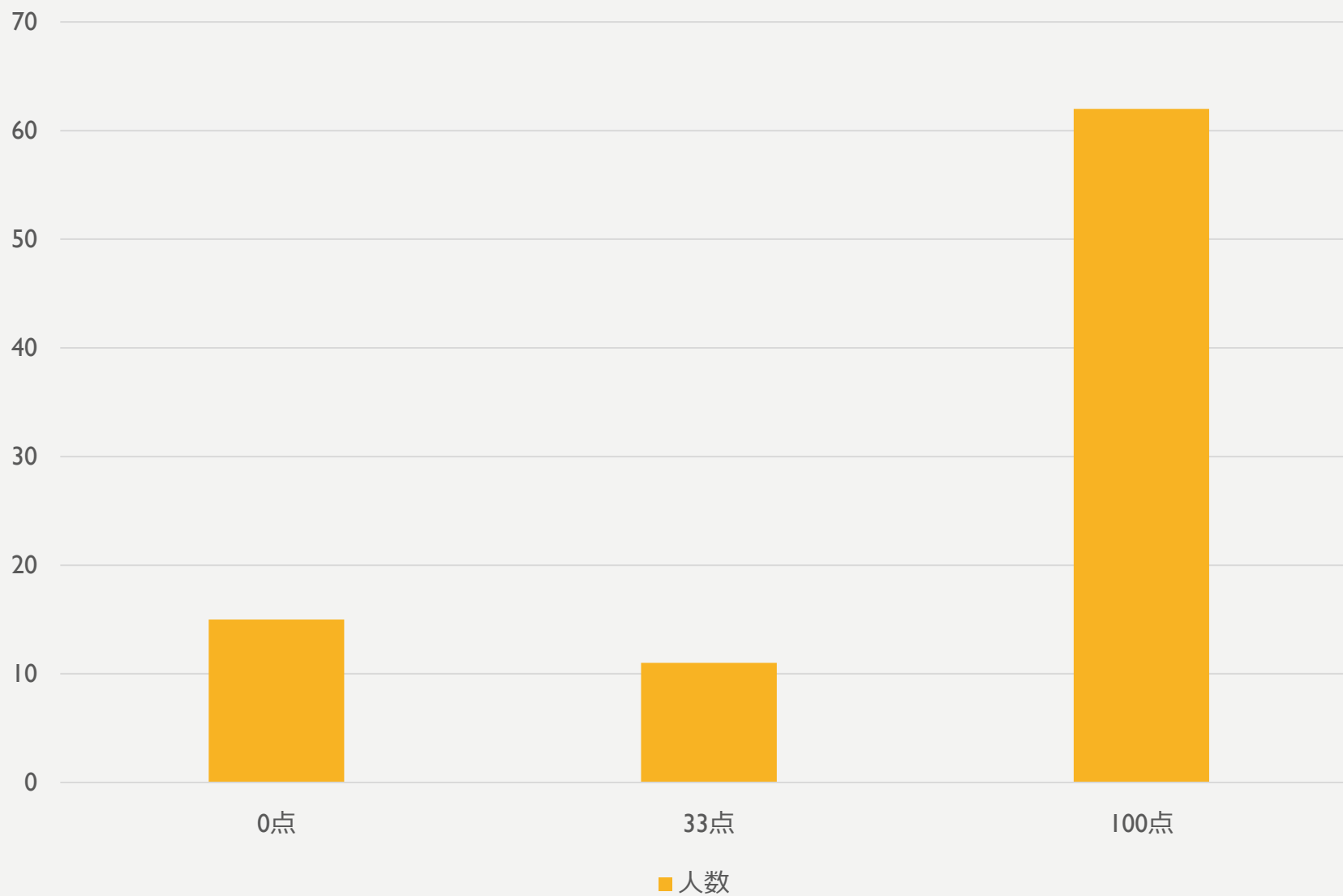
小課題 2 [67点]

- 区間を長さの昇順にソートする
- 風を順番に処理して V, L, R を更新しながら、まだ確定していない最も短い区間に注目しておく
- $L + R$ が注目している区間の長さを超えたら、その区間の配分を計算して、次に短い区間に注目する
- Q 回の風を 1 周見るだけですべての区間の配分が計算できる
- $O(Q + N \log N)$ で解ける

小課題 2 [67点]

- 解法2
- 長さ T の区間の配分は、 $L + R$ が T を初めて超えるタイミングでの「 L の値」, 「 R の値」, 「 L と R のどちらが増えたか」が分かれば計算できる
- よって各風が吹いたあとの上の3つの情報を配列に記録し、各 구간ごとに二分探索で超えるタイミングを求めればよい
- $O(Q + N \log Q)$

得点分布



問題3 集合写真

坂部圭哉

問題概要

- 長さ N の、 $1 \sim N$ を並び替えた数列が与えられます。
- 隣り合う要素を交換して、 $a[i] < a[i + 1] + 2$ を満たすようにするとき、交換回数の最小値は？

例

3	5	2	4	1
---	---	---	---	---



3	2	5	4	1
---	---	---	---	---



3	2	5	1	4
---	---	---	---	---

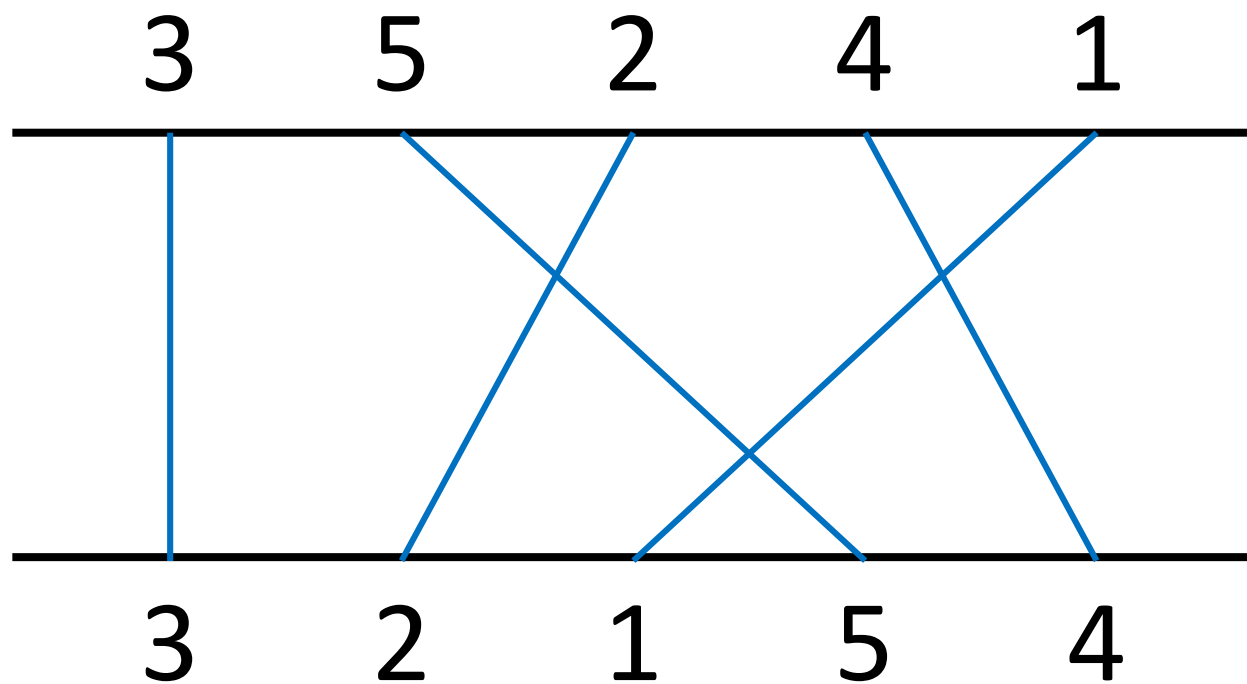


3	2	1	5	4
---	---	---	---	---

小課題1 ($N \leq 9$)

- 求めるもの: 「隣り合う要素を交換して目標の数列を作るときの、交換回数の最小値」
→ 目標の数列を1つ固定すると、これは **反転数** と呼ばれる数
- **反転数**
 - = 「バブルソートの交換回数」
 - = 「移動を線で表したときの交差回数」
 - = 「大小関係が逆転する 2 つのペアの個数」

小課題1 ($N \leq 9$)



交差が 3 箇所なので、反転数は 3

小課題1 ($N \leq 9$)

- 反転数は $O(N^2)$ で求められる
(バブルソートをしてもらい、大小関係が逆転するペアの個数を数えてもらい)
- $N!$ 通りのすべての数列を考えて、それぞれが条件を満たすかを判定して、反転数を求めれば良い。
(すべての数列を考えるには `next_permutation` という関数を使うと便利。)
- $O(N! N^2)$ で解けた。

小課題2 ($N \leq 20$)

- 先ほどは $N!$ 通りのすべての数列を考えていたが、条件を満たすのはそのうちのごく一部
→条件を満たす数列の性質が分かれば枝刈りできそう
- $a[i] < a[i + 1] + 2 \Leftrightarrow a[i + 1] \geq a[i] - 1$
「数字が小さくなるときは、1つずつしか小さくならない」

小課題2 ($N \leq 20$)

3	*	*	*	...
---	---	---	---	-----

↓ならば

3	2	1	*	...
---	---	---	---	-----

小課題2 ($N \leq 20$)

- このようにして考えると、目標の数列は、
「1, 2, ..., N」という数列を、いくつかの区間に
分けて、各区間内で反転したもの
という形になっている。

- 例:

3	2	1	5	4
---	---	---	---	---

 ←反転

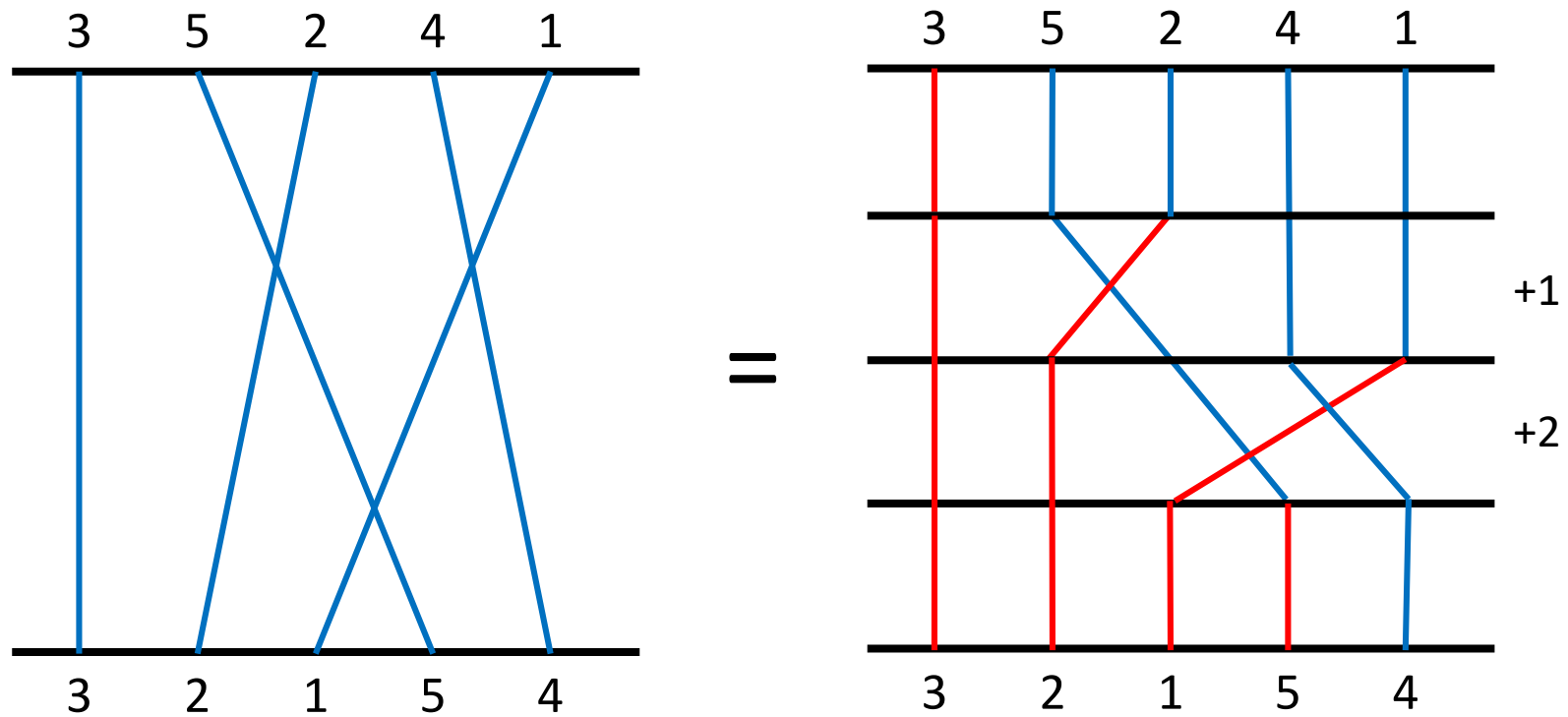
1	2	3	4	5
---	---	---	---	---

小課題2 ($N \leq 20$)

- 「1, 2, ..., N」という数列を、いくつかの区間に分ける方法: $2^{(N-1)}$ 通り
- $2^{(N-1)}$ 通りについて反転数を求める
- $O(2^N N^2)$ で解けた

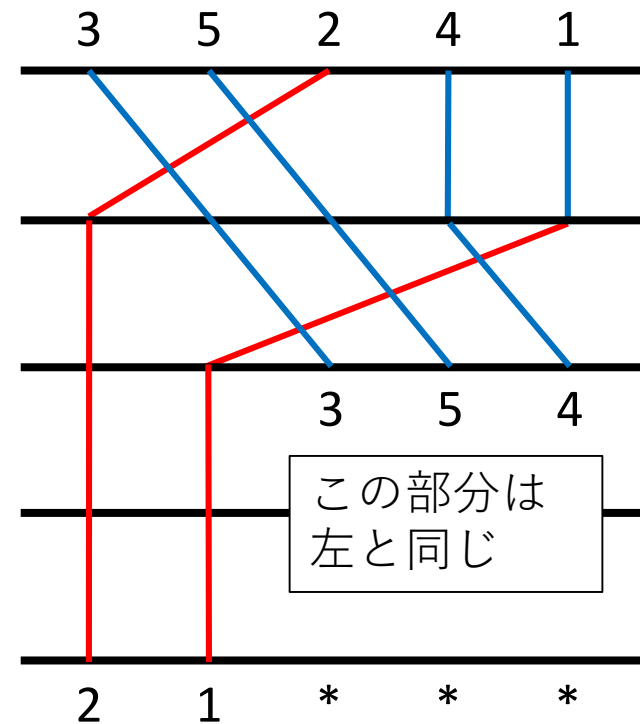
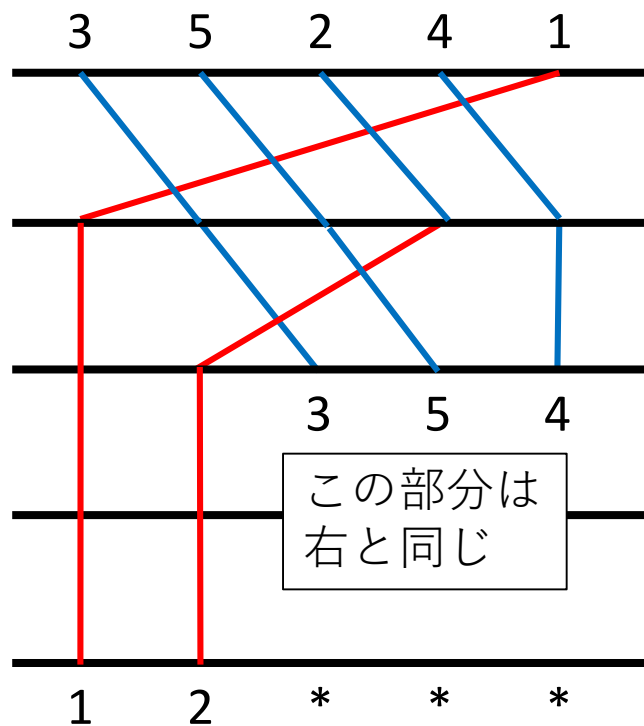
小課題3 ($N \leq 200$)

- 反転数を、左の要素から順に計算する



小課題3 ($N \leq 200$)

- 左の k 個が $1 \sim k$ の並べ替えになっているとき、 $k+1$ ステップより後の計算は同じ

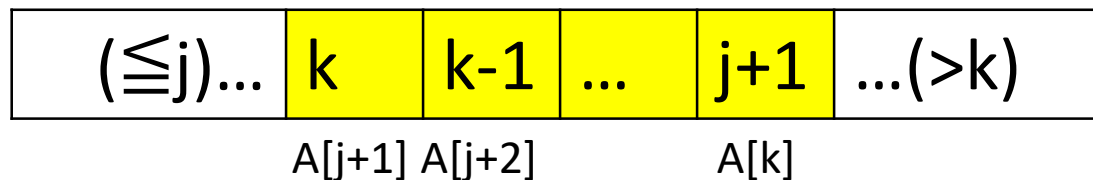


小課題3 ($N \leq 200$)

- 「左の k 個が $1 \sim k$ の並べ替えになっているときの k ステップまでの反転数の最小値」
という部分問題を繰り返せば解けそう
→**DP**

小課題3 ($N \leq 200$)

- $DP[k]$: 左の k 個が $1 \sim k$ の並べ替えになっているときの k ステップまでの反転数の最小値
- 左の k 個が $1 \sim k$ の並べ替えになっているとき、その数列は必ず以下の構造を取る



黄色の部分は
1 ずつ小さくなる

- $DP[j] + (k, k-1, \dots, j+1 \text{ の順にするときの反転数})$ を計算し、 j を動かしたときの最小値を $DP[k]$ とすればよい。

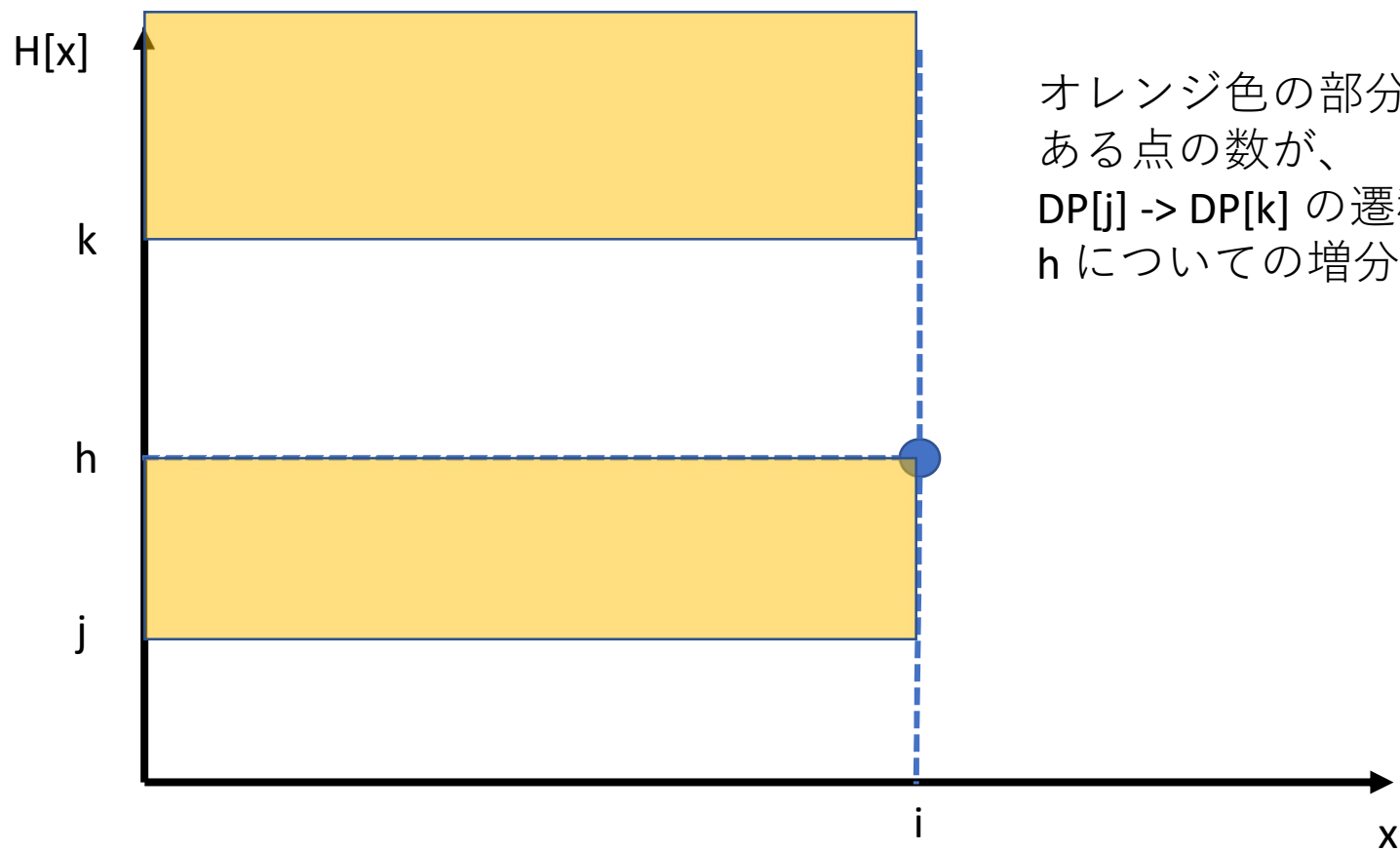
小課題3 ($N \leq 200$)

- 各反転数を $O(N^2)$ で求めれば、 $O(N^4)$ で解ける

小課題4 ($N \leq 800$)

- 先程の DP を改良する
- $DP[j] \rightarrow DP[k]$ への増分を考える。
それぞれの h ($j < h \leq k$) を左に持ってくるときの交換回数は、 $H[i] = h$ として、次の2つの和
 - $i' < i$ で $H[i'] > k$ となっている個数
 - $i' < i$ で $j < H[i'] \leq h$ となっている個数
- 図で書くと次のような感じ

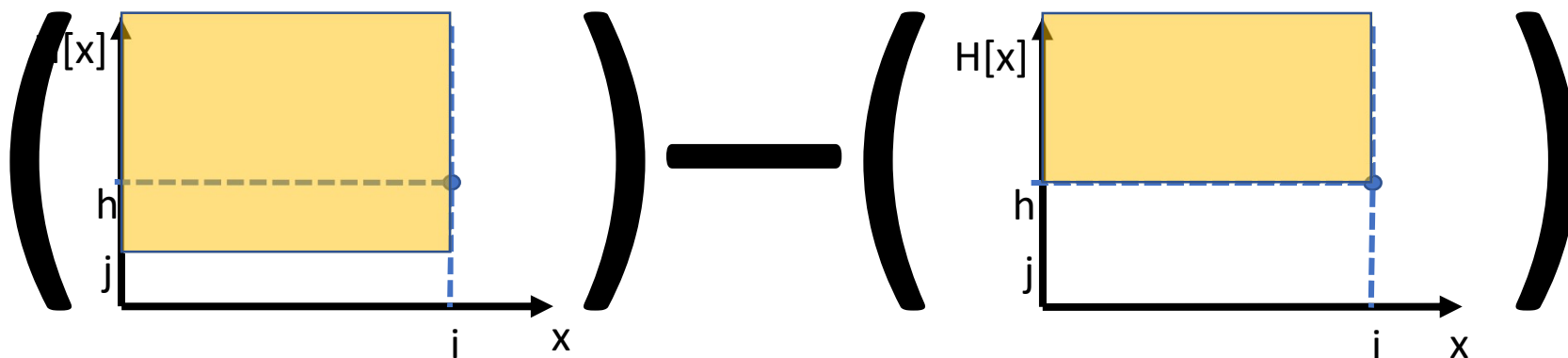
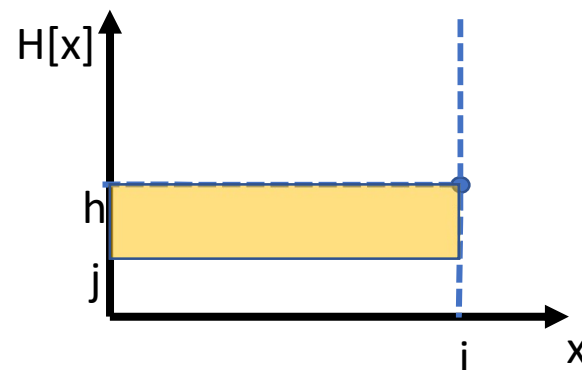
小課題4 ($N \leq 800$)



オレンジ色の部分にある点の数が、
 $DP[j] \rightarrow DP[k]$ の遷移で
 h についての増分

小課題4 ($N \leq 800$)

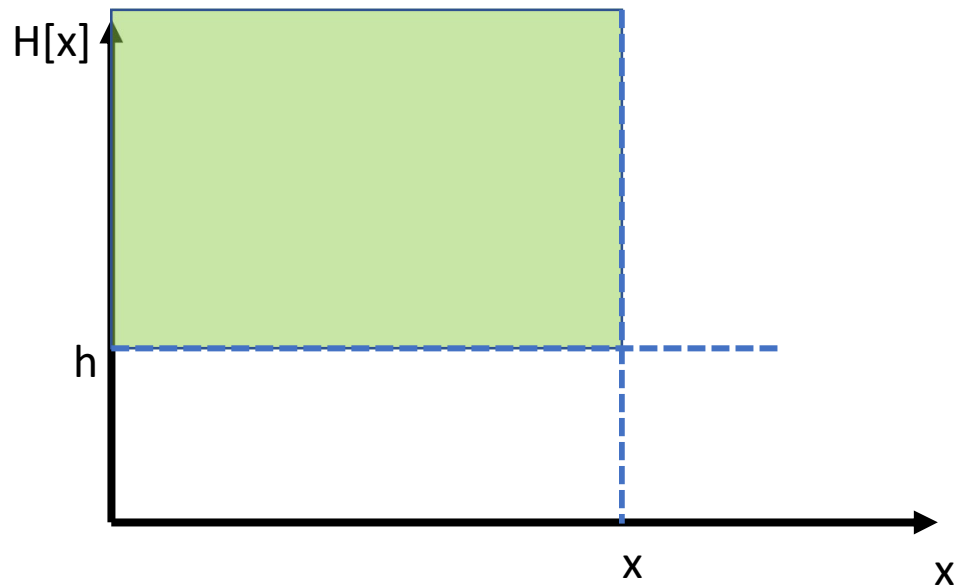
- 右図は下のように計算できる



小課題4 ($N \leq 800$)

- $\text{count}[x][h]$: 下図の緑領域の点の数
これを $O(N^2)$ で前計算しておけば、
 $O(N^3)$ で解ける。

※ BIT を使わなくても $O(N^2)$ でできます

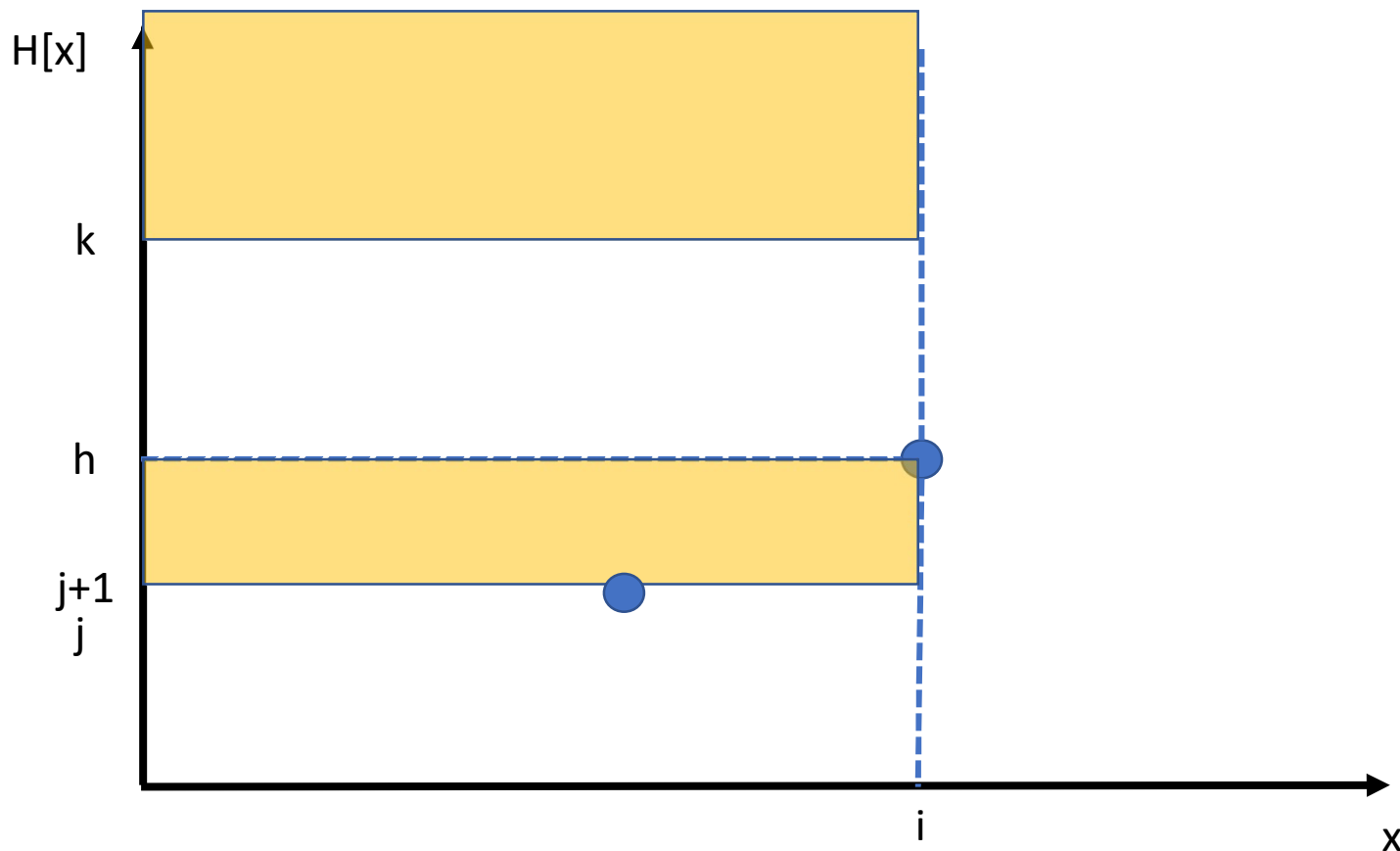


小課題5 ($N \leq 5,000$)

- $DP[j] \rightarrow DP[k]$ の遷移コストと
 $DP[j+1] \rightarrow DP[k]$ の遷移コストの差を考える
→もしこれが $O(1)$ で計算できれば、
 $O(N^2)$ となって解ける

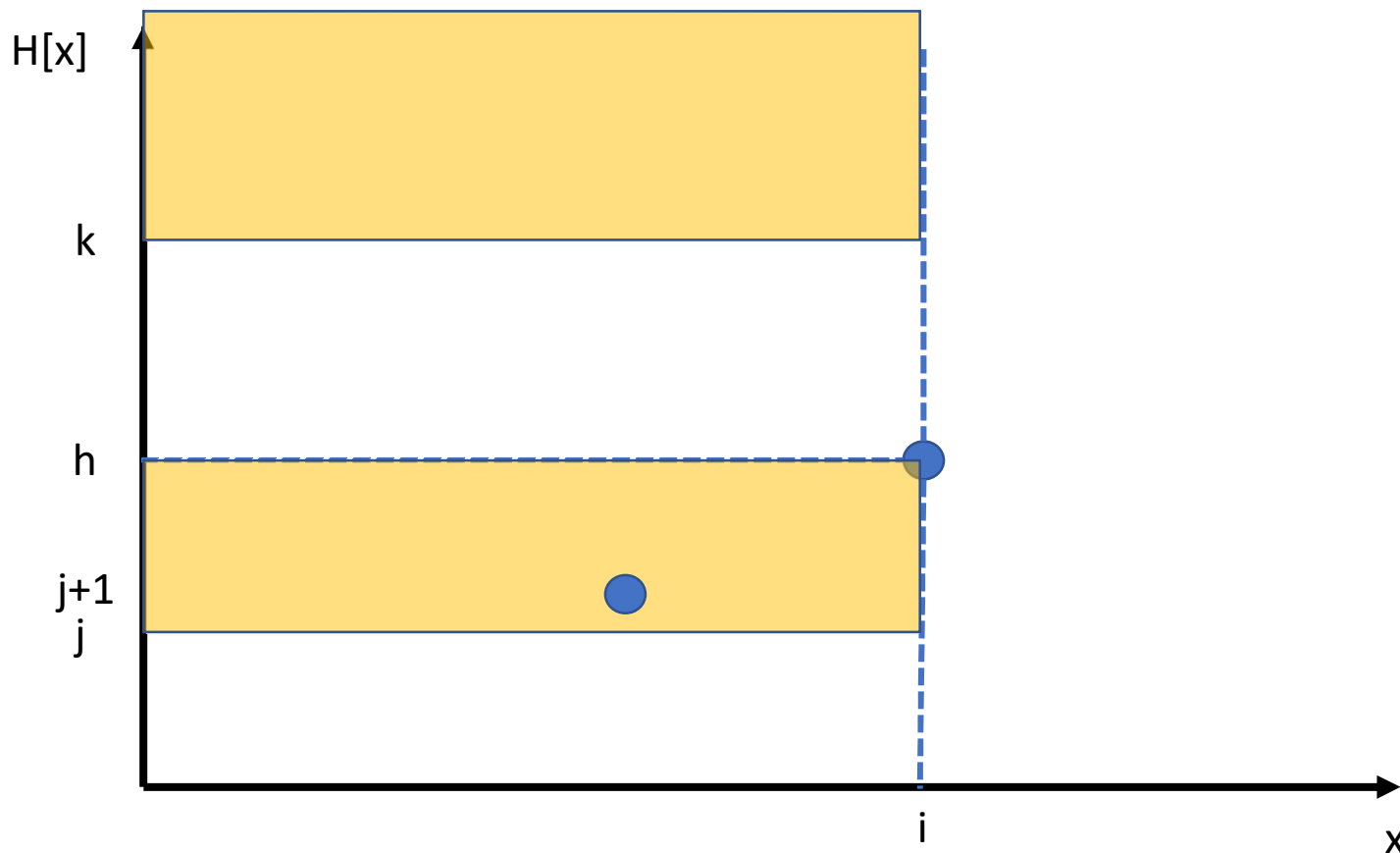
小課題5 ($N \leq 5,000$)

- $DP[j+1] \rightarrow DP[k]$ のコスト (h について和を取る)



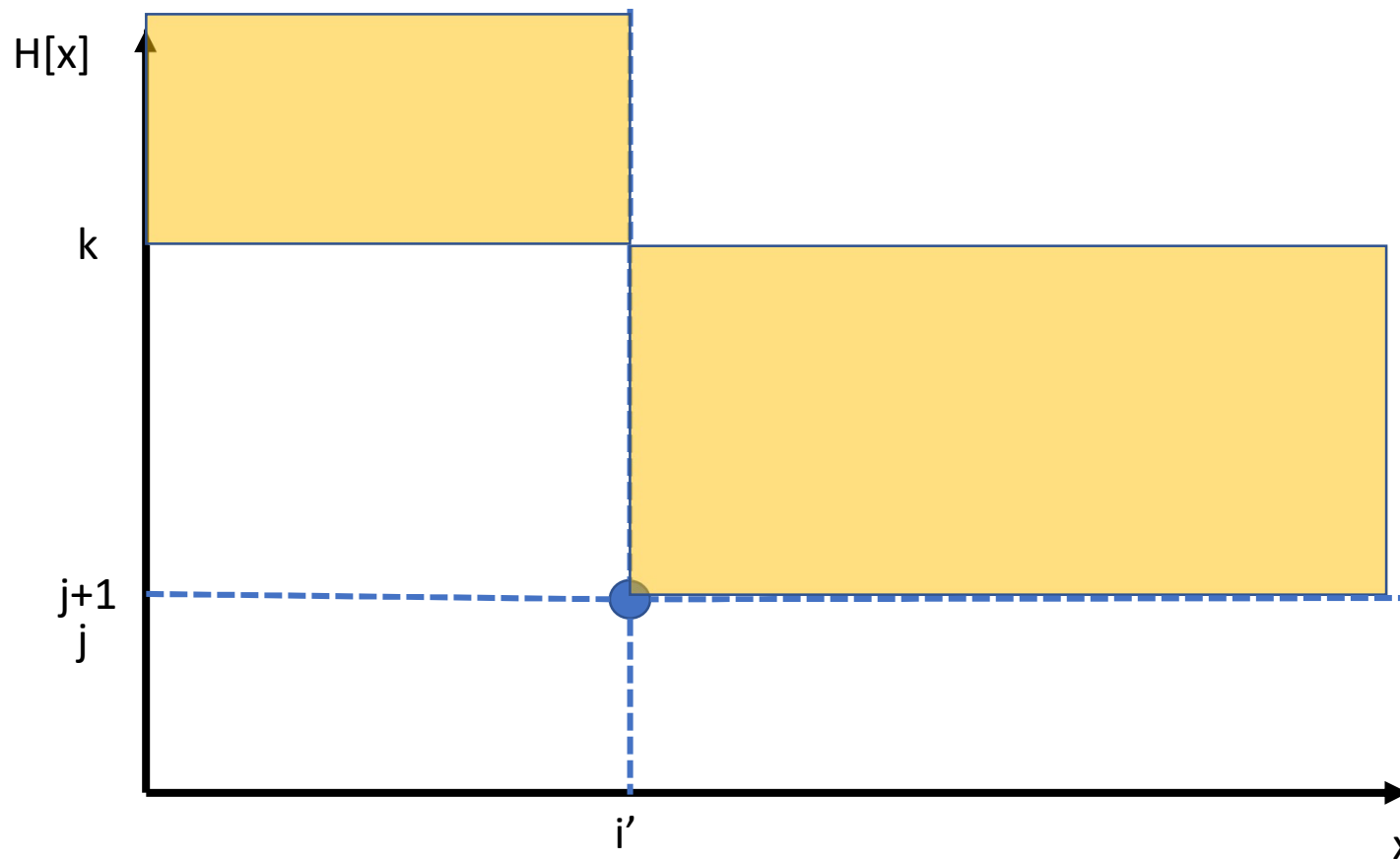
小課題5 ($N \leq 5,000$)

- $DP[j] \rightarrow DP[k]$ のコスト (h について和を取る)



小課題5 ($N \leq 5,000$)

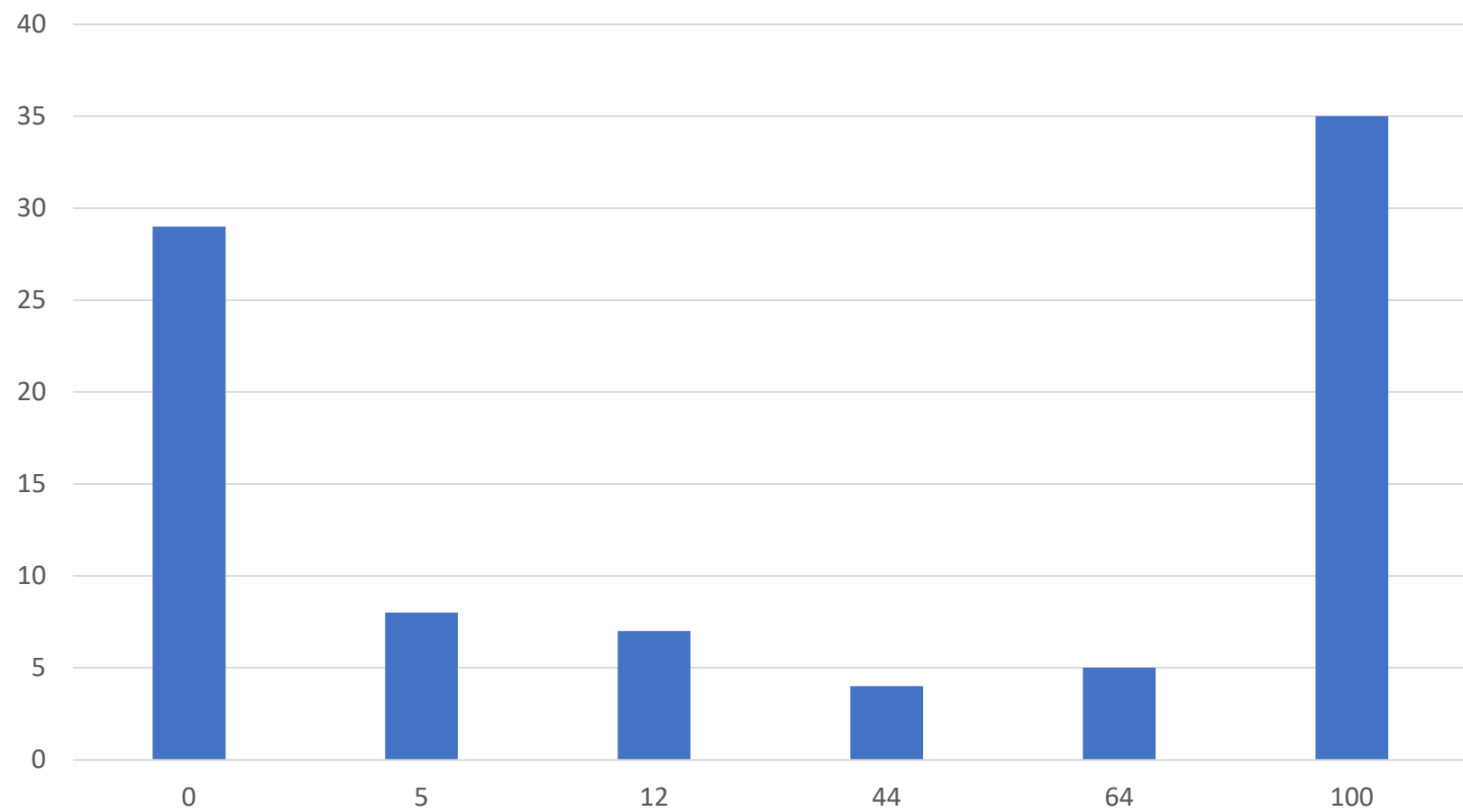
- これら 2 つの差分



小課題5 ($N \leq 5,000$)

- $O(N^2)$ となって解けた！！

得点分布



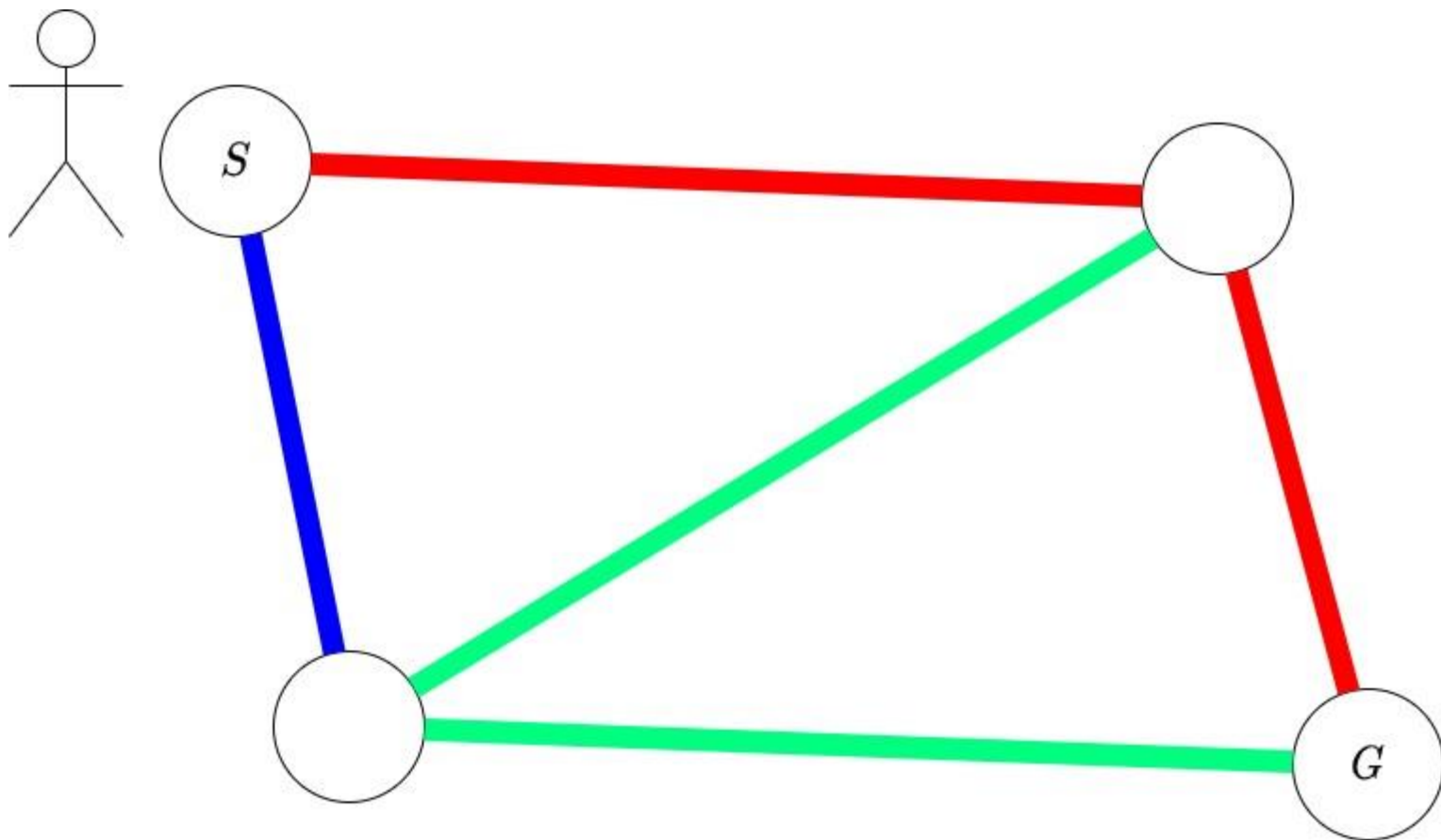
JOI 2020/2021 本選 4

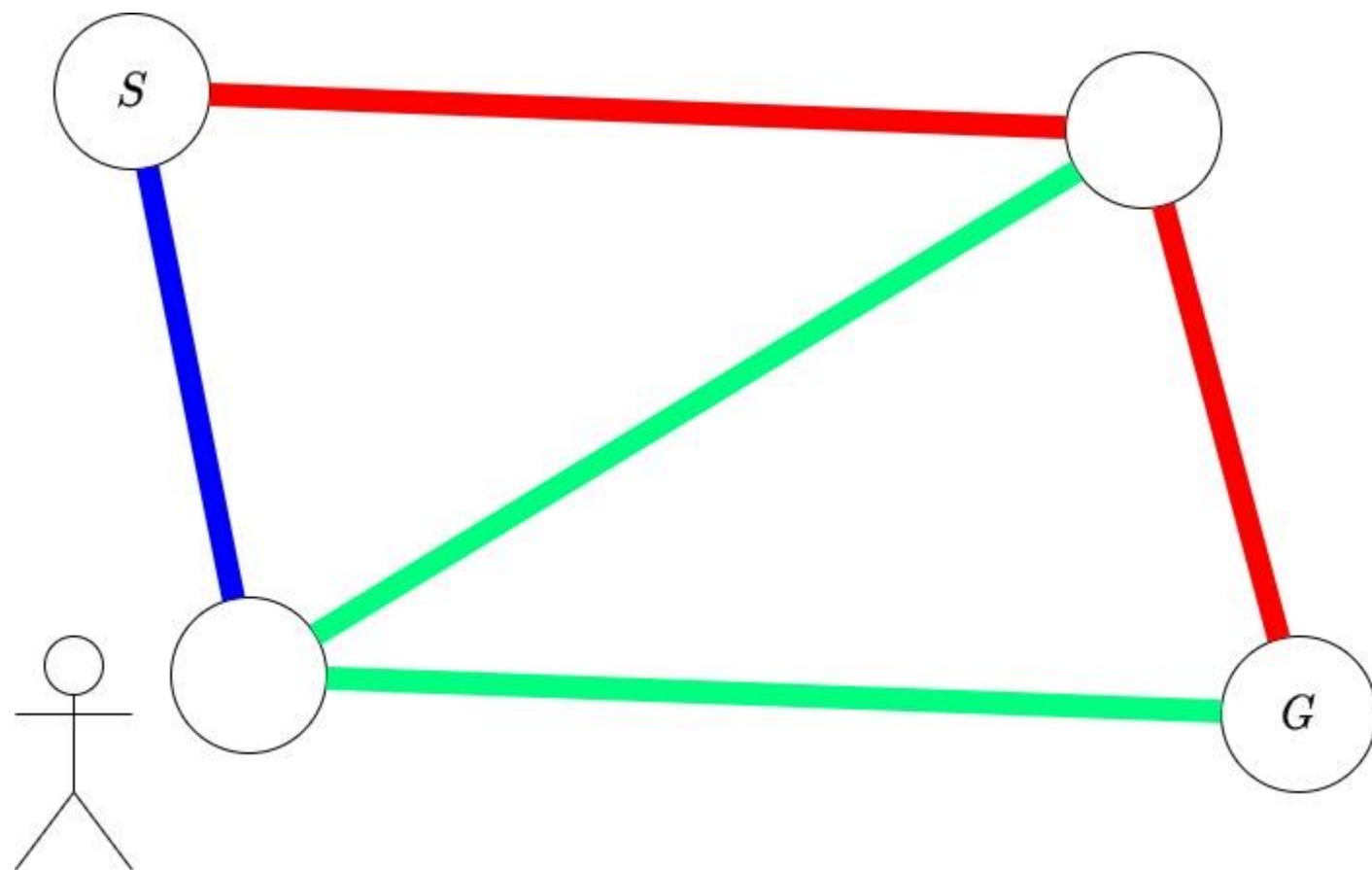
ロボット (Robot)

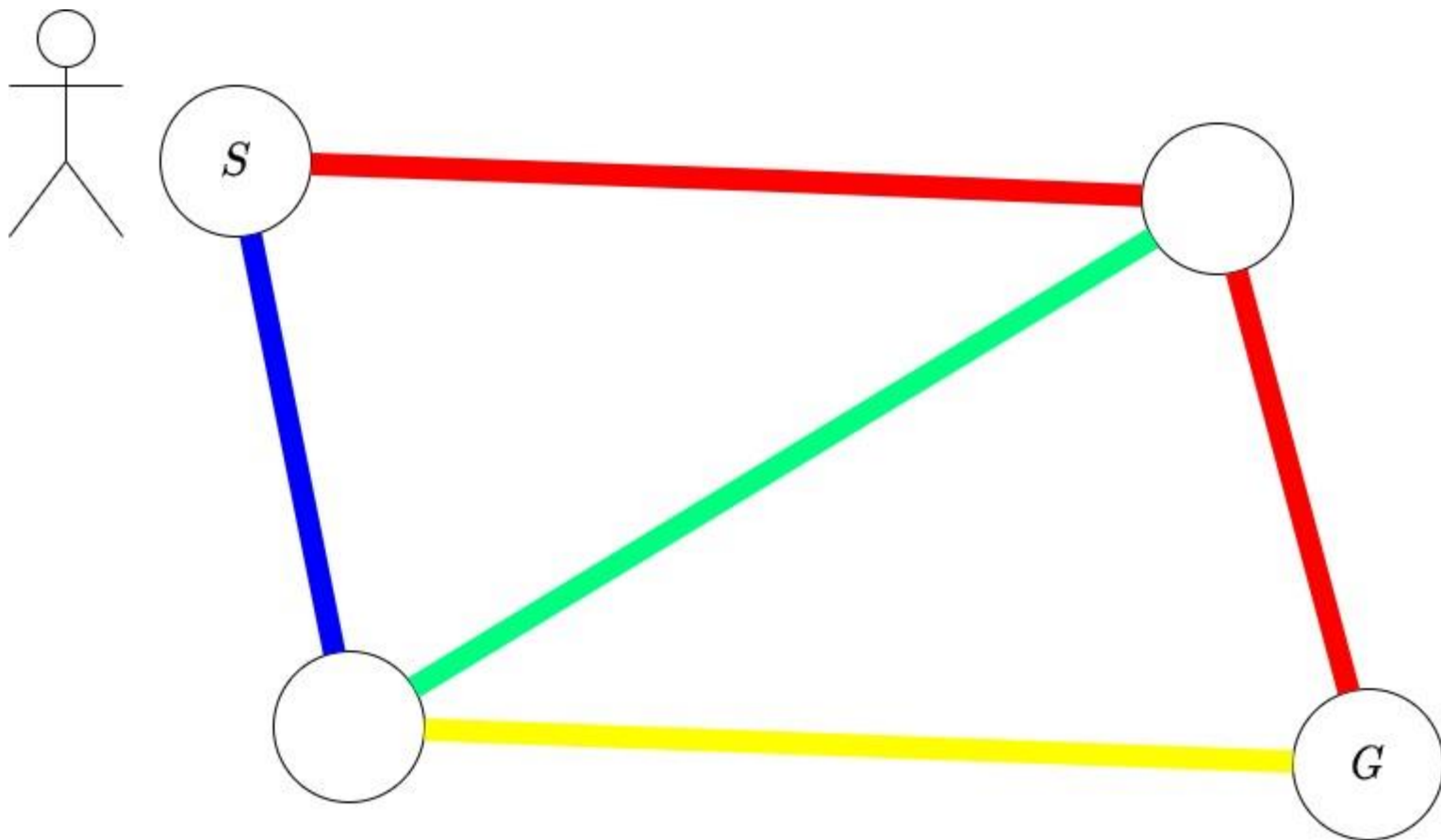
解説: 木ノ下 恭範

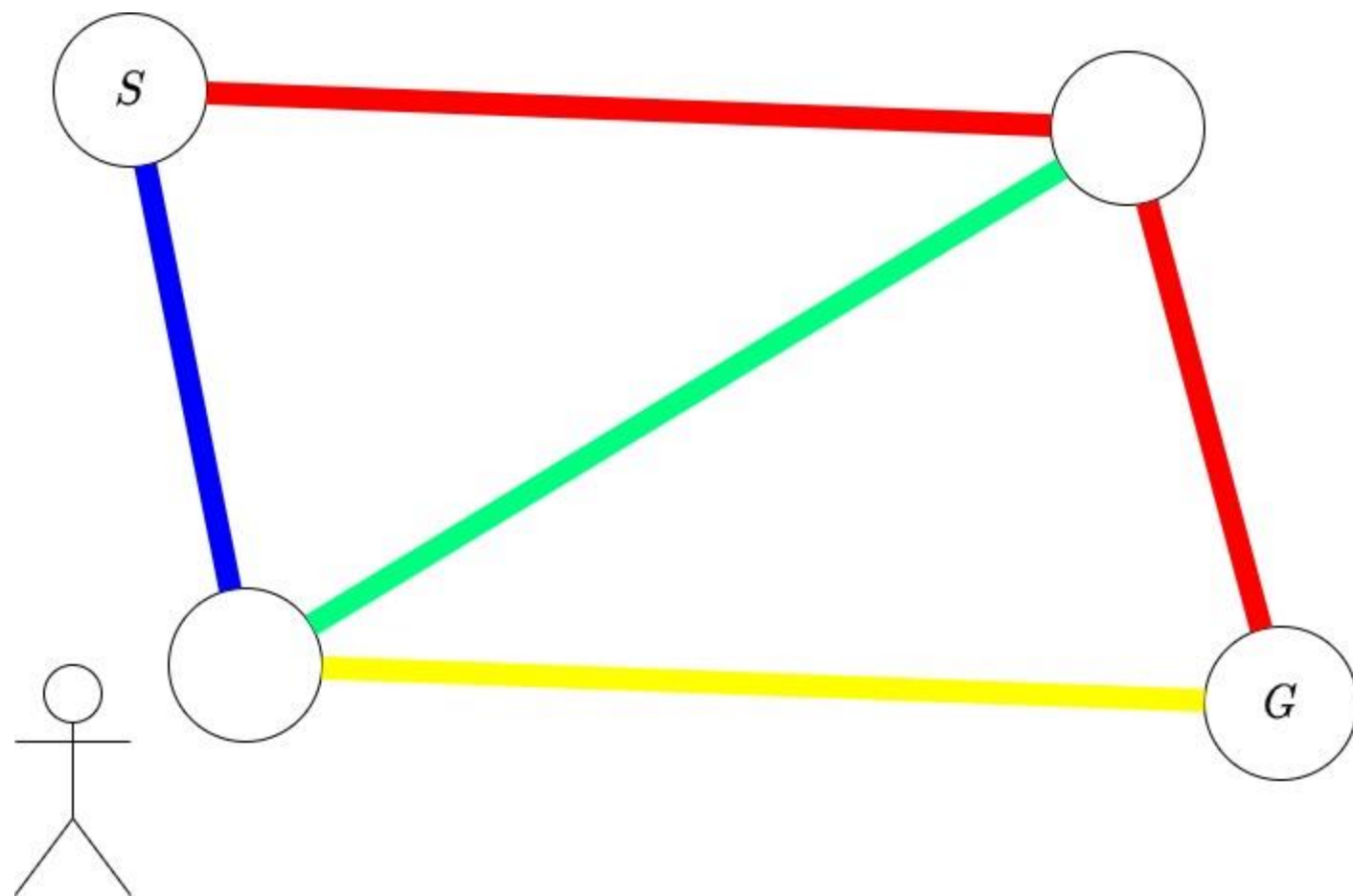
問題概要

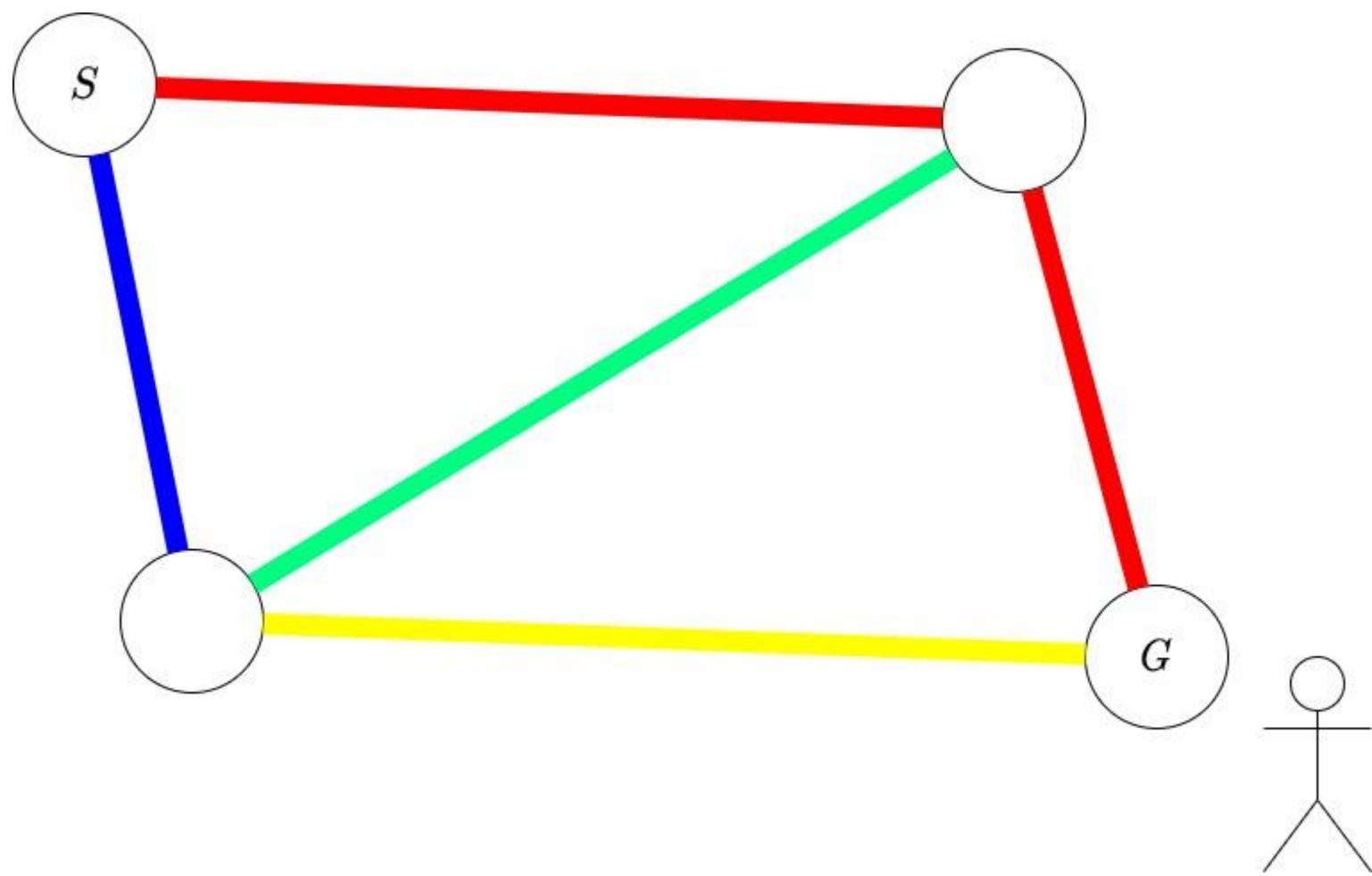
- 辺に色のついた単純無向グラフが与えられる。
- ロボットに色を指示すると、「ロボットがいる頂点から、その色の辺が丁度 1 本伸びている」ときに限り、その辺を選んで進む。
- ロボットを 1 から N に届けられるように、事前にいくつかの辺を塗り替えたい。
- それぞれの辺について、塗り替えに掛かる費用が決まっているので、費用の合計を最小化してください。











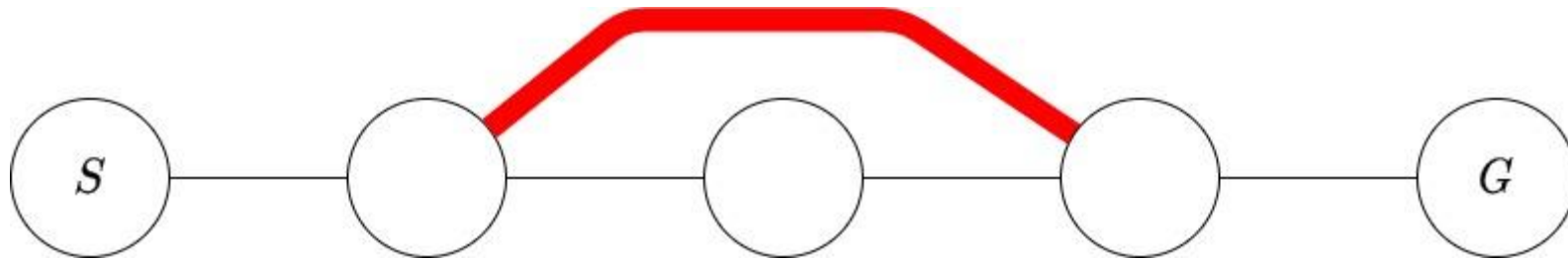
観察

- 色は M 色あるので、必ず、他のどの辺とも異なる色に塗り替えるとしてよい。
- 全ての辺を塗り替えれば、 1 と N が非連結な場合以外、常に移動可能。

観察

- 色は M 色あるので、必ず、他のどの辺とも異なる色に塗り替えるとしてよい。
- 最適な塗り替え方の上で 1 から N への最短経路を考える。
- これは同じ点を二度通らないパスになる。
- パス上の頂点に接続していない辺は塗り替えていない (塗り替えても得るものが無いため)。
- パス上で距離が 2 以上離れた点同士をつなぐような辺は、塗り替えられてない。

- 他のだの辺とも異なる色に塗り替えるとしてよい。
- 最適な塗り替え方の上で 1 から N への最短経路を考える。
- パス上で距離が 2 以上離れた点同士をつなぐような辺は、塗り替えられてない。：もしそうだとすると、他と異なる色なのでこの辺は自由に通ることが出来るため、より短い経路が見つかってしまう。



観察

- 最適な塗り替え方の上で 1 から N への最短経路を考える。
- これは同じ点を二度通らないパスになる。
- パスの頂点に接続していない辺は塗り替えていない (塗り替えても得るものが無いため)。
- パス上で距離が 2 以上離れた点同士をつなぐような辺は、塗り替えられてない。

→ パスを通りながら、必要になってから色を塗り替えるとしてよい。さらに、今までに塗り替えた辺は、直前の頂点と今いる頂点を結ぶ辺以外覚えなくてよい。

小課題 1

- $N \leq 1000, M \leq 2000$
- ある頂点から辺を選んで進むとき、大きく分けて 2 つの方法がある。
 - X: 進みたい辺を塗り替えて進む。
 - Y: 進みたい辺以外で同じ色の辺を全て塗り替えて進む。
- 今までに塗り替えた辺は、パス上で直前の 1 本しか気にしなくてよいことに注意すると、この 2 つ以外は考えなくてよい。
- (問題文のグラフとは異なる) グラフ上での最短路問題に帰着することが出来る。

小課題 1

- (頂点 v に居る, 辺 e が塗られている) を頂点とするグラフで、最短距離を求めればよい。
- 塗られている辺によって、移動 Y のコストが安くなる可能性があることに注意。
- 頂点の数は $N+2M$, 辺の数は $(N+2M)N$ 程度になる
- Dijkstra 法を用いると $O(NM \log(NM))$

小課題 2

- $P = 1$
- 0 円で進めないのなら、移動 X を使ってしまえばよい。次に移動するとき手前の辺が塗られているのを活用できる可能性がある分、移動 X は移動 Y より優秀であるため。
- 移動 X を行うなら、直前に何が塗られていたかは忘れても良い。移動 Y を行うときも、塗られている辺と進みたい辺の色が同じときだけが問題である。

小課題 2

- 移動 X を行うなら、直前に何が塗られていたかは忘れても良い。移動 Y を行うときも、塗られている辺と進みたい辺の色が同じときだけが問題である。
- (v に居る, e が塗られている) から (v に居る, どこも塗られていない) にコスト 0 の辺を張ると、(v に居る, e が塗られている) からは、 e と同じ色の辺への移動 Y だけを考えればよい。
- さらに、 e と同じ色の辺が v に 3 本以上接続しているなら、移動 Y はしない。よって、移動 Y は高々 1 つ。

小課題 2

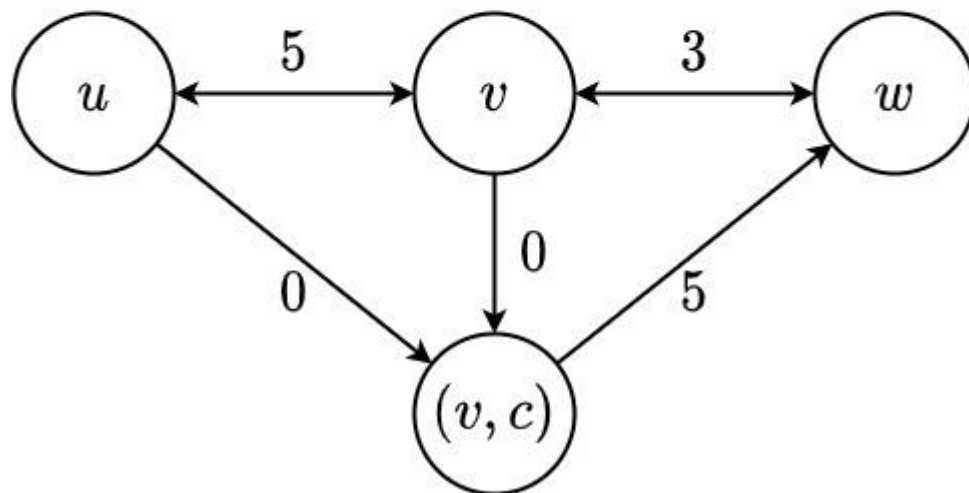
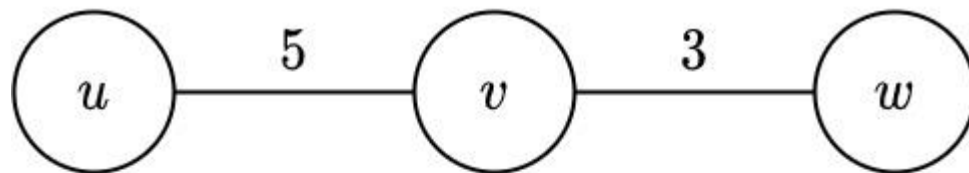
- 頂点の数 $N+2M$, 辺の数 $4M+2M+2M$ 程度のグラフ上で最短距離を求めればよい。
- $O(M \log(M))$

小課題 3

- (再掲) 直前に塗られた辺が必要になるのは、同じ色の移動 Y を行う場合のみである。
- $u \rightarrow v$ (移動 X), $v \rightarrow w$ (移動 Y) という遷移を、全ての u に対して上手くまとめ上げる。
- $u \rightarrow v$ の移動 X をコスト 0 ということにしてしまい、 $v \rightarrow w$ の移動 Y において uv を塗ったことを忘れて普通にコストを計算すれば、つじつまがあう。
- (v に居る, 次は色 c についての移動 Y を行う) という頂点を追加し、 u からコスト 0 で遷移させればよい。

小課題 3

- (v に居る, 次は色 c についての移動 Y を行う) という頂点を追加し、 u からコスト 0 で遷移させればよい。



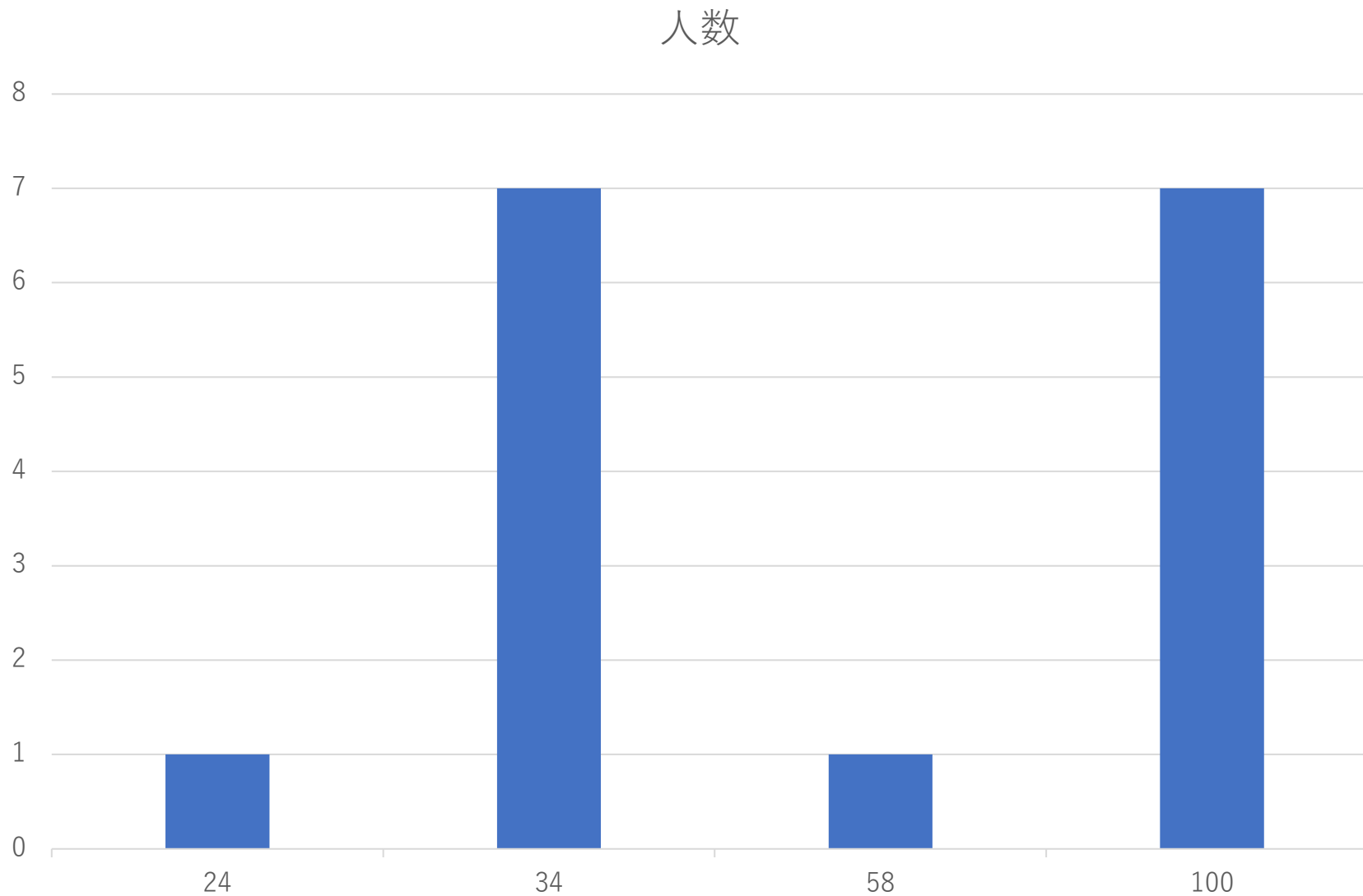
小課題 3

- (v に居る, 次は色 c についての移動 Y を行う) という頂点を追加し、 u からコスト 0 で遷移させればよい。
- 頂点の数は $N+2M$ 以下、辺の数は $2M+4M$ 程度になる。
- $O(M \log(M))$
- 全体を通して、番号の振りにくい頂点を上手く管理する必要があります。
- C++ なら `std::map<std::pair<int, int>, int>` などを利用すると良いかもしれません。

小課題 3 (別解)

- 頂点 v に接続する色 c の辺のうち、重みが上位 2 つに入っていない辺へは、移動 Y を行う意味がない。一番重い辺が直前に塗られていたとしても、なお移動 X の方が安い。ため。
- 小課題 1 で作成した拡張グラフの辺を削減する。
- $(v$ に居る, e を塗った) から $(v$ に居る, 塗られた辺なし) にコスト 0 で辺を張る。
- $(v$ に居る, e を塗った) からは、 v に接続する e と同じ色の辺のうち、重み上位 2 つへの移動 Y を行う。
- 辺の本数が線形になり、十分高速。

得点分布



Dungeon 3 解説

担当 : 高谷 悠太 (yutaka1999)

問題概要

- $N + 1$ 層からなるダンジョンがある.
- M 人のプレイヤーがある層からある層へ移動しようとしている.
- 各プレイヤーが必要とするコインの枚数を求める.

問題概要

- プレイヤーには体力が定まっている.
 - 初期値は 0
 - 上限値はプレイヤーごとに異なる.
- どの層でも体力を 1 回復させられる.
 - 回復にはコインが必要.

問題概要

- 体力はある層から次の層へ進む際に消費する.
- 一つ前の階層に戻ることはできない.

入出力

- A_i $\coloneqq$ 第 i 層から第 $i + 1$ 層へ進む際に消費する体力
- B_i $\coloneqq$ 第 i 層で回復するのにかかるコイン枚数

入出力

- S_j $\coloneqq$ プレイヤー j が現在いる層
- T_j $\coloneqq$ プレイヤー j の目標の層
- U_j $\coloneqq$ プレイヤー j の体力の上限

到達可能性

- S から T へ到達できる条件を考える.
- $M := A_S$ から A_{T-1} までの最大値
- $M \leq U$ が必要十分.
 - これは RMQ でクエリ毎 $O(\log N)$ の時間で可能.
- これ以降常に到達可能とする.

小課題 1

- 制約： $N, M \leq 3\,000$
- 各プレイヤーごとに時間 $O(N)$ かけてよい.

小課題 1

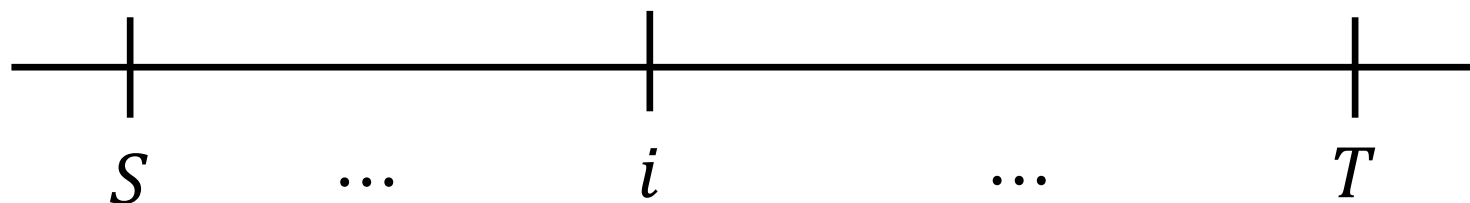
- 選択が生じるのは各階層での回復量のみ.
- U が十分大きければ

体力が 0 になったときに今まで訪れた階層のうちコストが最小の泉で回復したことにする

のが最適.

小課題 1

- ダンジョンを数直線として表す.



- 座標を $X_i := A_1 + \dots + A_{i-1}$ とする.

小課題 1

- 第 i 層で得られる体力を用いて移動できる領域は線分 $[X_i, X_i + U]$
- 各 $S \leq i < T$ について線分 $[X_i, X_i + U]$ を単位長さあたりコスト B_i で用いることができる.
- 線分 $[X_S, X_T]$ の被覆にかかる最小コストが求める値.

小課題 1

- 線分 $[X_s, X_T]$ の被覆にかかる最小コストが求める値.
- 各線分 $[x, x + 1]$ ごとの最小コストを足し合わせればよい.
 - 線分 $[x - U + 1, x]$ 内にあるダンジョンでの B の最小値がコスト.

小課題 2

- 制約： U が一定
- 各階層に対応する線分 $[X_i, X_i + U]$ がプレイヤーによらない.

小課題 2

- プレイヤーを S の降順に処理する.
- S が小さくなるごとに使える線分が増え, 各線分 $[x, x + 1]$ の最小コストも更新される.
- コストが変化する区間は `stack` 等で求まる.

小課題 2

- 更新 : 区間代入
取得 : 区間 sum
- これは segment tree で答えられる.

小課題 3

- 制約： T が常に $N + 1$
- U の変動は最小コストにどう影響するか？

小課題 3

- 各 $S \leq i < T$ について，単位長さあたりコスト B_i の線分 $[X_i, X_i + U]$ を用いる長さと U の関係調べる.
- $L_i := B_L \leq B_i$ なる最大の $L < i$
 - 存在しない場合は -1 とし， $X_{-1} := -\infty$ とする.
- $R_i := B_R < B_i$ なる最小の $R > i$
 - 存在しない場合は $N + 1$

小課題 3

- 最適解において、線分 $[X_i, X_i + U]$ は $\max(X_i, X_{L_i} + U)$ から $\min(X_i + U, X_{R_i})$ まで用いられる.
 - ただし、縮退している場合は使われない.
- つまり、線分 $[X_i, X_i + U]$ を被覆に用いる長さは U についての折れ線で表せる.
 - 折れ線とは、区分的に一次的な関数.

小課題 3

- プレイヤーを S の降順に処理する.
- 各 $S \leq i \leq N$ について対応する U についての折れ線を考え, それらの総和を保持する.
 - U が与えられたときに対応する値を答えられるようにする.
- S を小さくしたときの更新について調べる.

小課題 3

- 各 i について
 - R_i は不変.
 - L_i は -1 から S へと変更しうる.
- L_i が更新される i は **stack** 等で求まる.
- L_i の更新と同時に対応する折れ線も更新する.

小課題 3

- 以下のクエリに帰着される.
 - 節点が定数個の折れ線を加算する
 - ある座標での値を取得する
- これは BIT 等で可能.
- 時間計算量は $O((N + M) \log(N + M))$

小課題 4

- 制約：追加の制約はなし
- 実は $T = N + 1$ の場合に帰着できる.

小課題 4

- $C :=$ 線分 $[X_S, X_{N+1}]$ の被覆の最小コスト
 $D :=$ 線分 $[X_T, X_{N+1}]$ の被覆の最小コスト
- 答えは $C - D$ “くらい”のはず.

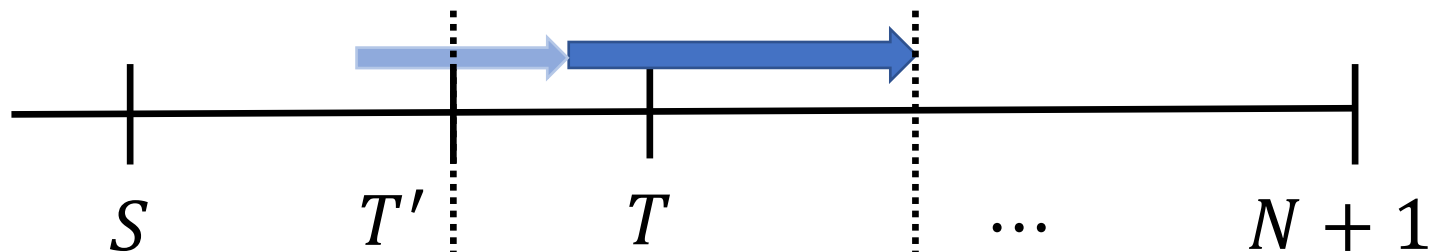
小課題 4

- 線分 $[X_s, X_{N+1}]$ を最小コストで被覆する際に、座標 X_T を被覆しているのが線分 $[X_{T'}, X_{T'} + U]$ 由来の線分であるとする.
- T' は線分 $[X_T - U, X_T]$ 上にある階層のうち、 B が最小のもの.

小課題 4

- 線分 $[X_T, X_{N+1}]$ の最小コストでの被覆は，初期位置が S でも T' でも変わらない．

初期位置が S の場合



初期位置が T' の場合



小課題 4

- $C :=$ 線分 $[X_S, X_{N+1}]$ の被覆の最小コスト
 $D' :=$ 線分 $[X_{T'}, X_{N+1}]$ の被覆の最小コスト
- 答えは $C - D' + (X_T - X_{T'}) \times B_{T'}$
- これで小課題 3 に帰着された.